VISUAL NOTES • COMPLETE GUIDE

Private Local AI for an Obsidian Vault

Turning My Obsidian Vault Into a Local AI Engineer

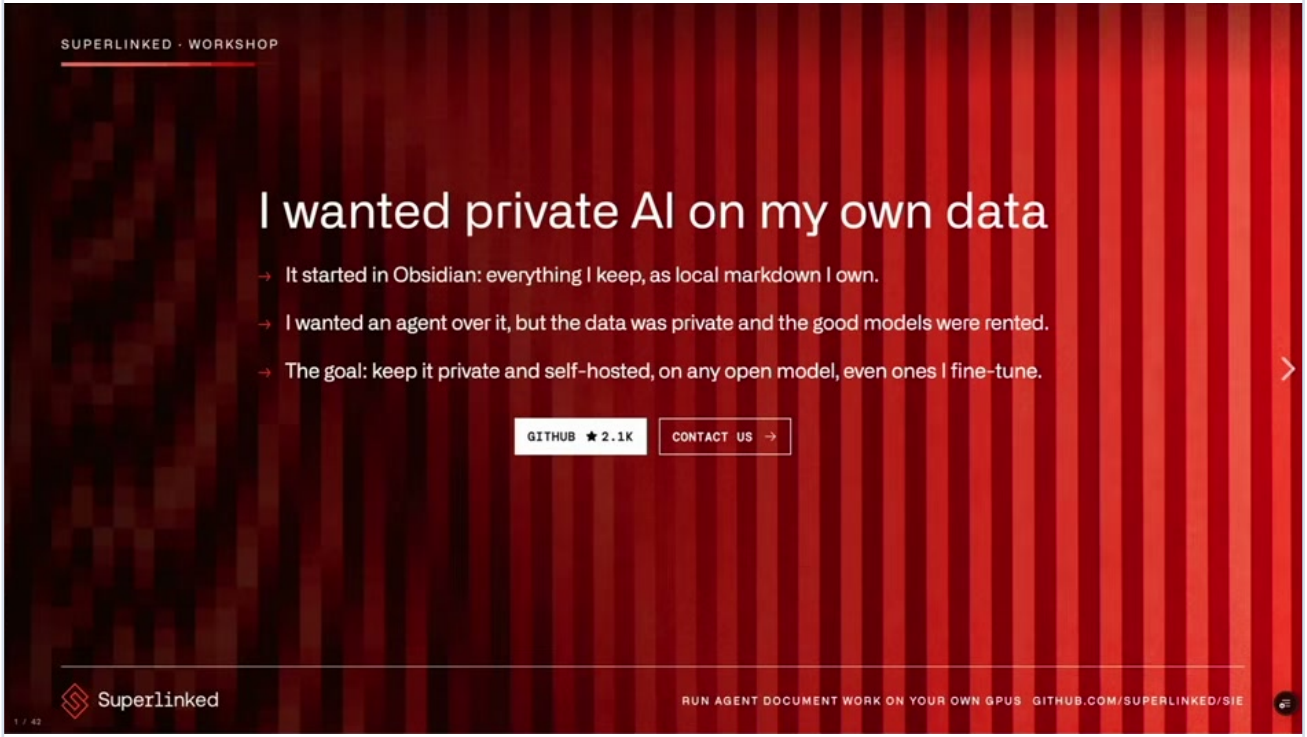
47:25

- 40 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=Ndp5VaqFrl0

Private AI over locally owned notes

The starting requirement is control, not merely better organization



NOTES

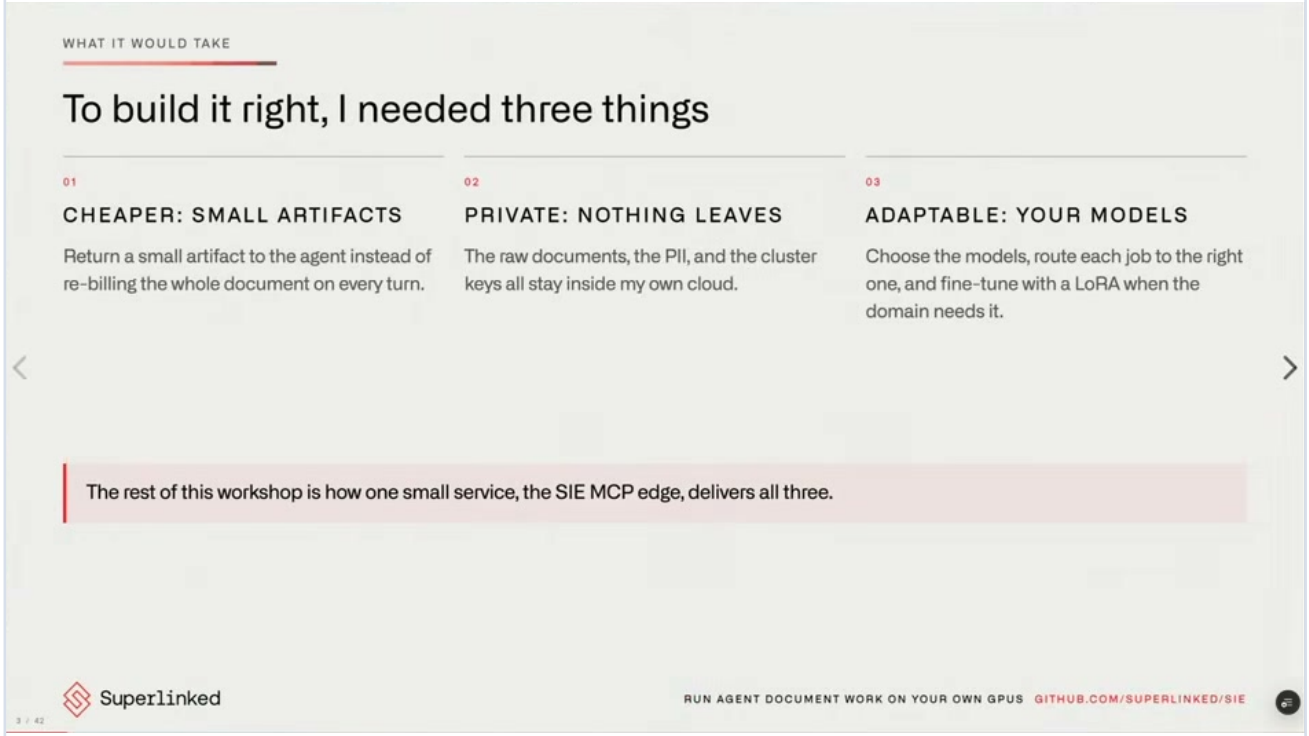
- Obsidian keeps the source material as local Markdown, but an external agent can still expose those files when given broad access.
- The target system must preserve a familiar agent workflow while moving sensitive document work into infrastructure the user controls.
- The privacy boundary matters most for institutions and workflows containing personally identifiable information, contracts, or internal records.

CORE IDEA

Local storage is not private once an external agent receives the raw files.

Three design requirements

Cheaper artifacts, private processing, and adaptable models



NOTES

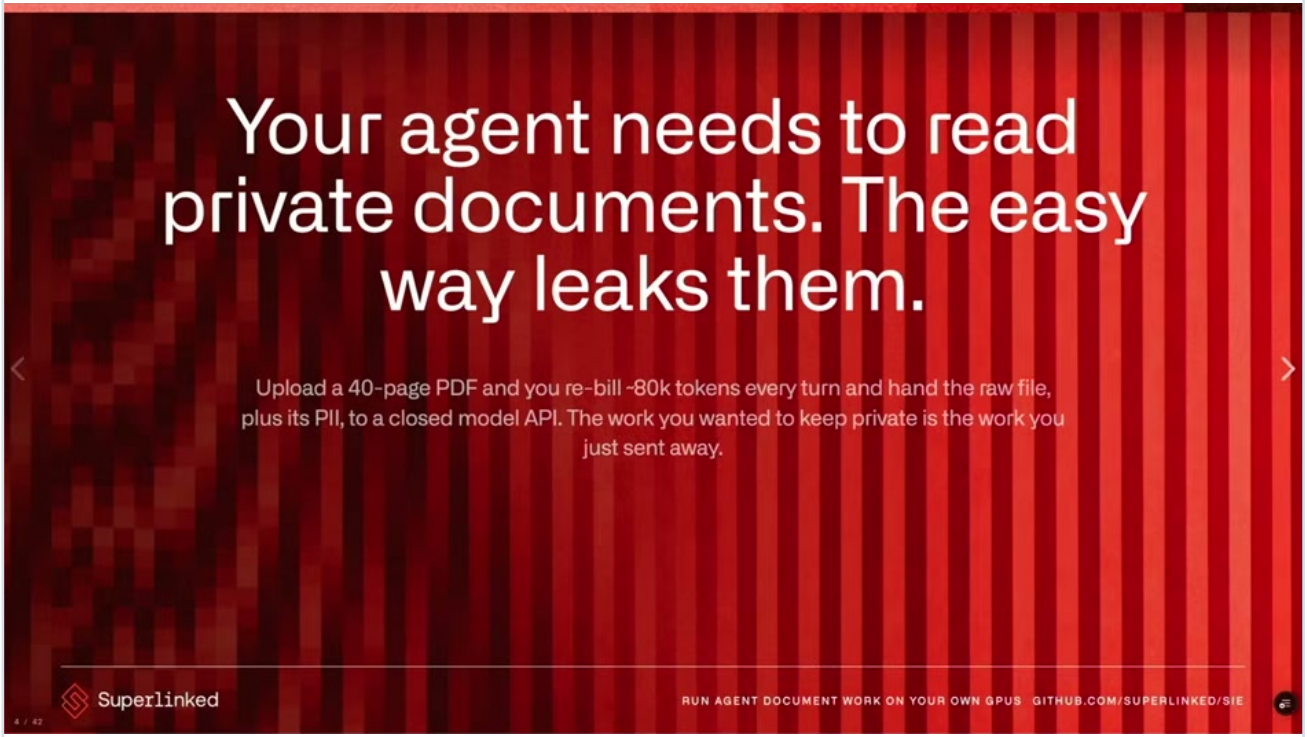
- Return compact artifacts instead of repeatedly billing full documents, scans, images, and formatting into the agent context.
- Keep raw documents, personally identifiable information, and cluster credentials inside the user's own cloud or virtual private cloud.
- Allow task-specific model selection and low-rank adaptation (LoRA), rather than binding every job to one hosted model.

CORE IDEA

The architecture must improve cost, privacy, and model choice together.

The easy document path leaks data

Directly uploading long private documents creates both exposure and token cost



NOTES

- The simplest workflow sends the complete document to a cloud model, including content irrelevant to the requested task.
- Scans and image-heavy PDFs amplify cost because optical character recognition and layout understanding consume substantial context.
- A local preprocessing boundary can reduce what crosses to the external agent, but its security still depends on correct tool and infrastructure design.

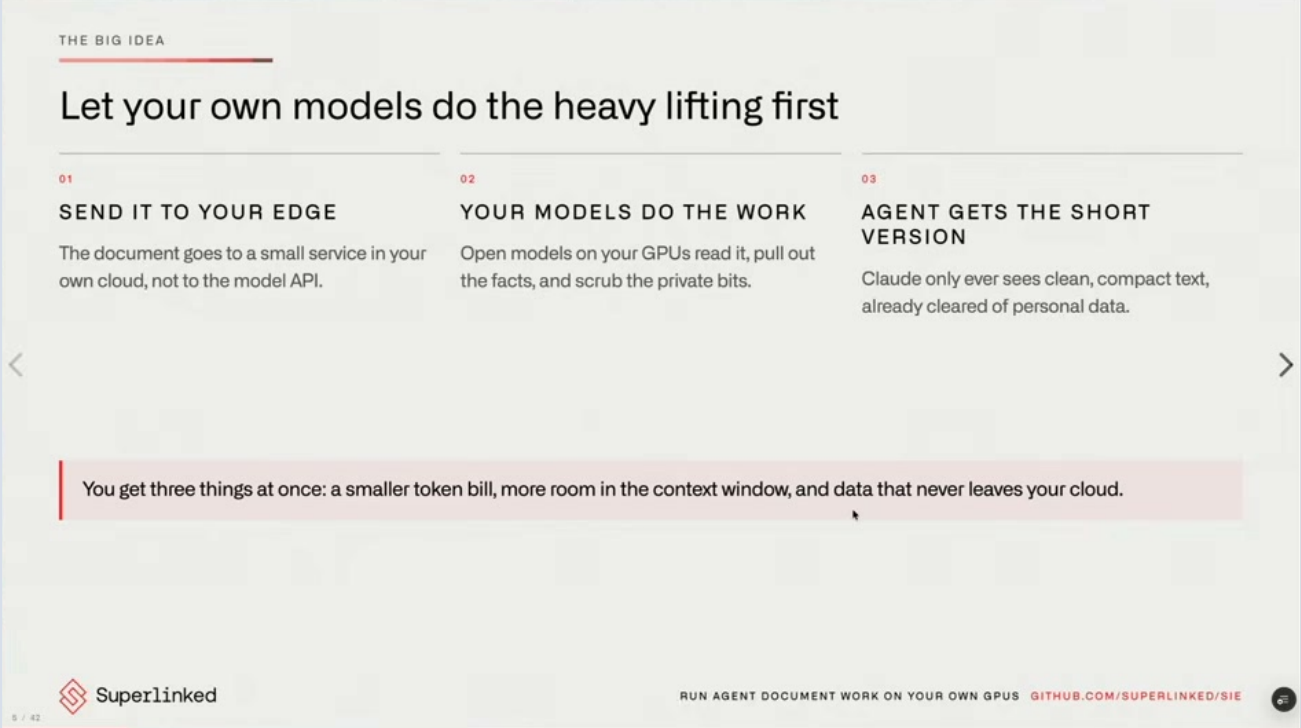
CORE IDEA

Convenient document upload can expose more data than the agent actually needs.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Let local models do the heavy lifting

The agent receives a compact artifact rather than the raw source



NOTES

- The agent sends a task to an edge service deployed in the user's environment instead of processing the source document itself.
- Open models perform optical character recognition, extraction, tagging, summarization, or redaction near the protected data.
- Only a cleaned Markdown artifact or structured answer returns to the agent, leaving more context-window capacity for reasoning and follow-up work.

CORE IDEA

Preprocess privately, then expose only the minimum useful result.

Token savings come from smaller artifacts

The claimed reduction is illustrative, not an independent benchmark



NOTES

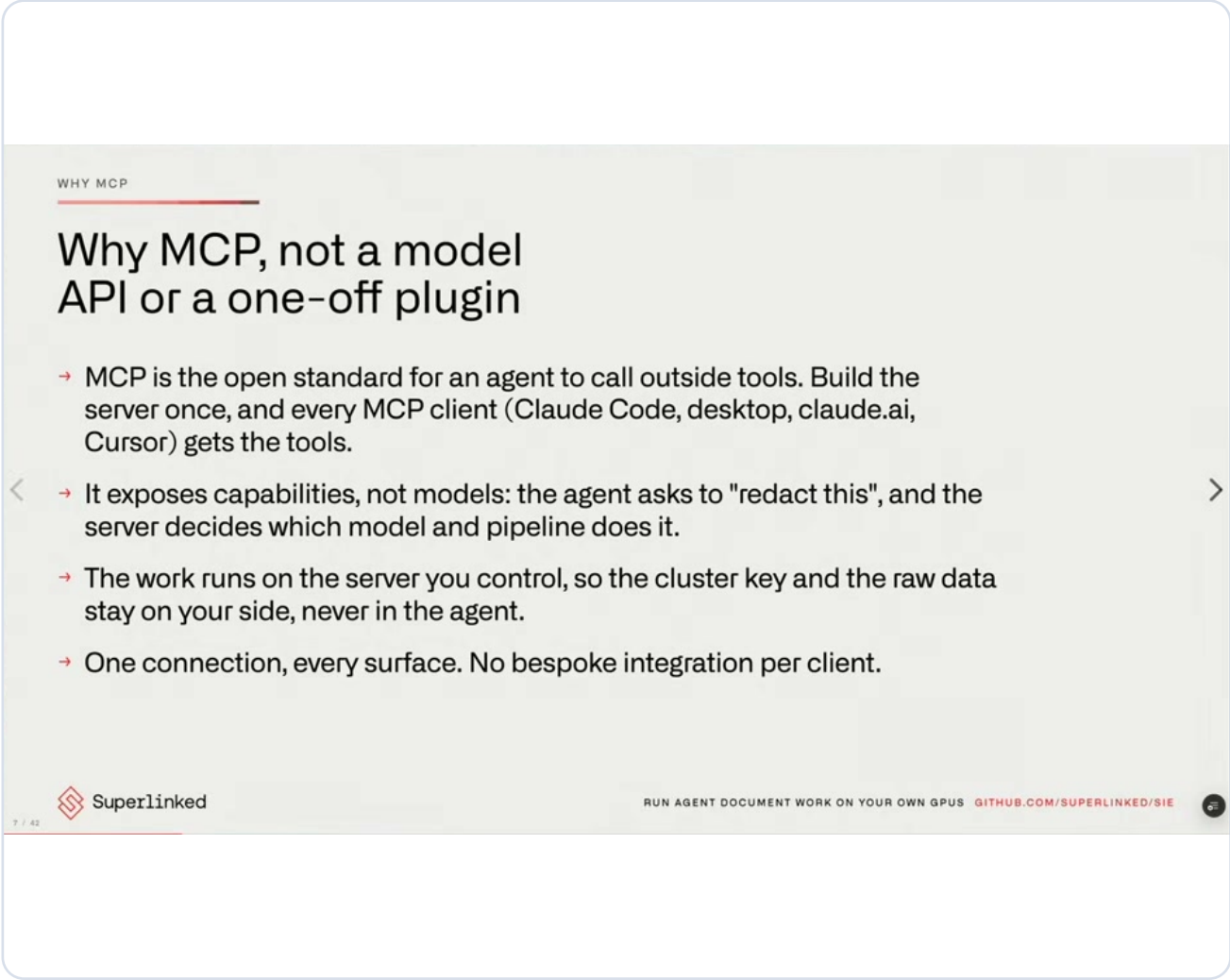
- Sending full pages, images, and formatting consumes far more tokens than returning compact, cleaned Markdown.
- The talk uses an approximate fifteen-percent artifact size to explain the mechanism; actual savings depend on document type and task.
- Smaller context also reduces distraction and context loss, not just API charges.

CORE IDEA

Compression improves both economics and the signal available to the main agent.

Why use Model Context Protocol

MCP exposes capabilities while keeping execution behind a controlled boundary



NOTES

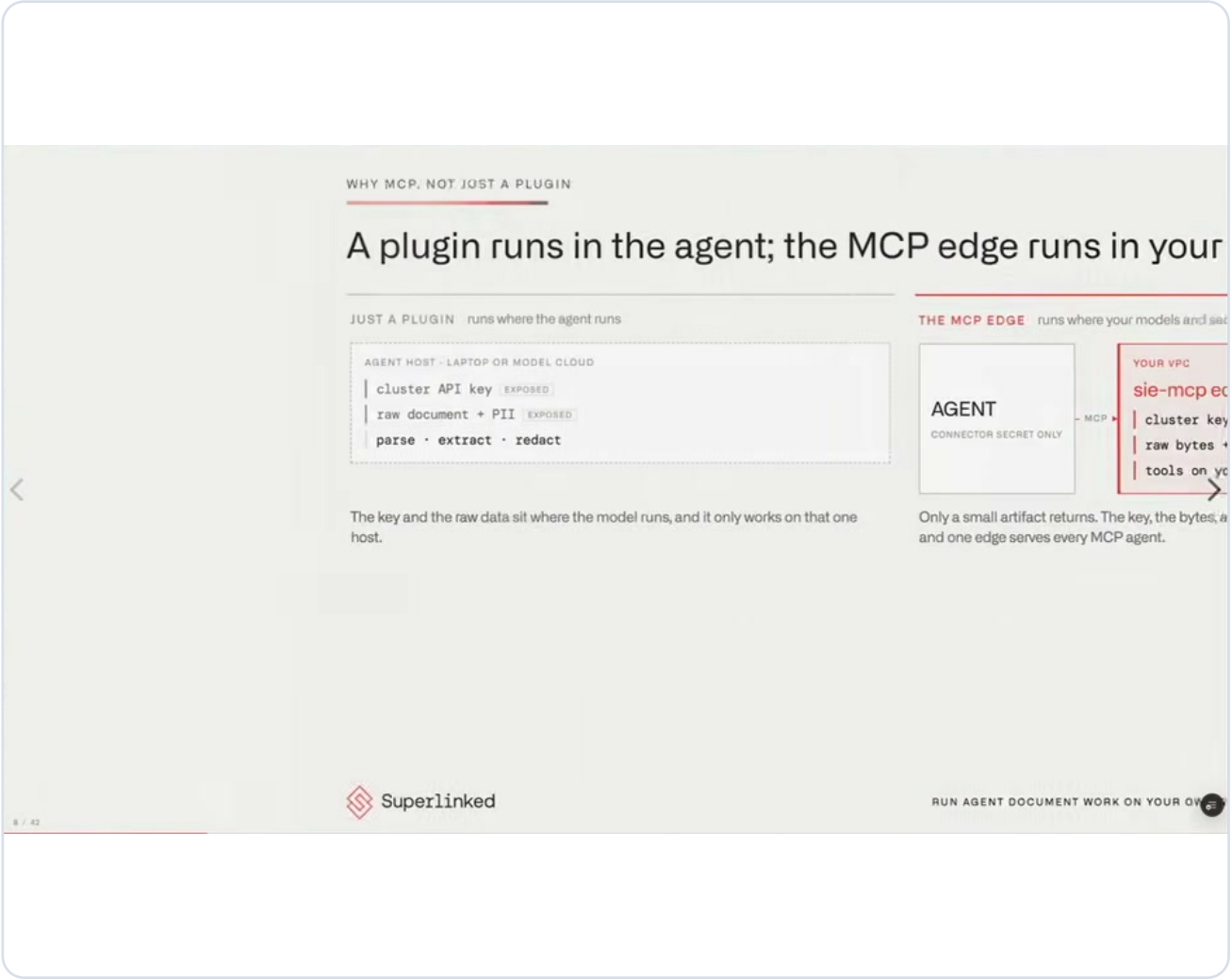
- Model Context Protocol (MCP) gives agents a standard interface for calling external tools without embedding every implementation inside the agent.
- The server chooses the pipeline and model, while raw data and cluster keys remain on controlled infrastructure.
- One server can support several compatible clients, avoiding a separate plugin and credential path for each application.

CORE IDEA

MCP separates agent ergonomics from protected execution and model infrastructure.

Plugin execution versus a cloud edge

The edge moves tool logic and sensitive work out of the agent process



NOTES

- A conventional plugin often runs alongside the agent on the user's device and remains coupled to that application.
- An MCP edge offers the same callable experience while executing document work in the user's cloud or virtual private cloud.
- This central service becomes a reusable backbone for adding tools, policies, routing, and operational controls.

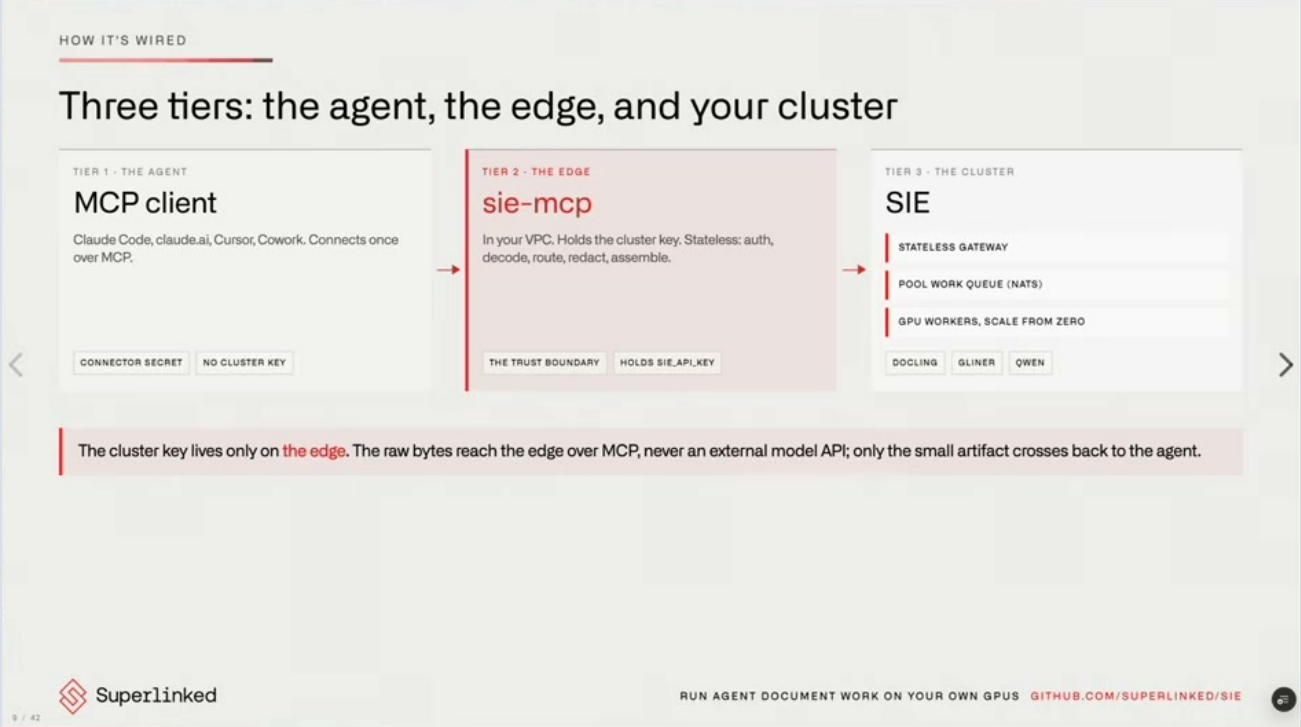
CORE IDEA

The edge is a durable service boundary, not another client-specific extension.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Three-tier private inference architecture

Client, MCP edge, and GPU-backed inference cluster



NOTES

- The MCP client lives in an agent host such as a coding or desktop assistant and issues capability calls.
- The MCP edge authenticates requests, retains cluster credentials, and converts calls into jobs for the inference service.
- The inference cluster supplies worker pools, GPUs, queues, and multiple open models; only the resulting artifact returns across the boundary.

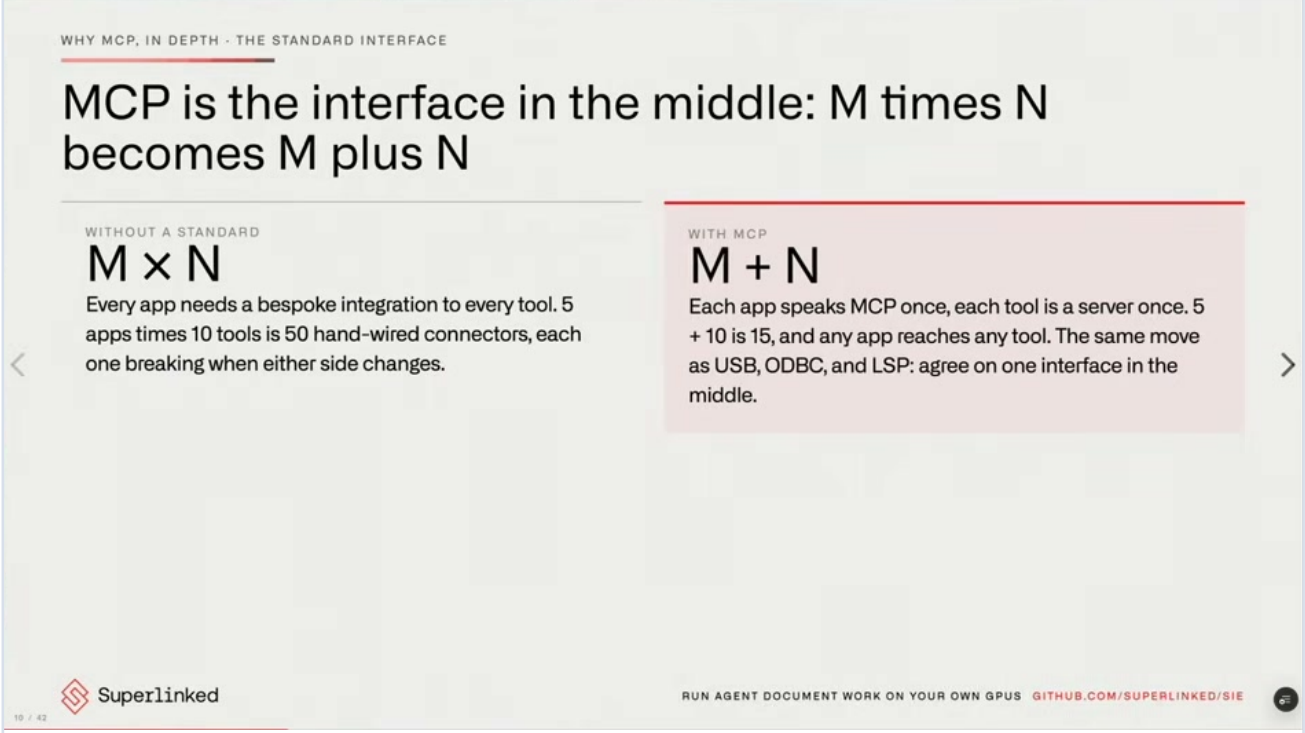
CORE IDEA

Each tier has one responsibility and a clear security boundary.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

A common interface changes integration scaling

M × N bespoke links become M + N standard implementations



NOTES

- Without a standard, every application needs a custom integration for every external tool, creating a multiplicative maintenance burden.
- With MCP, each application implements the client side once and each tool implements the server side once.
- The analogy is a shared interface standard such as USB, ODBC, or the Language Server Protocol.

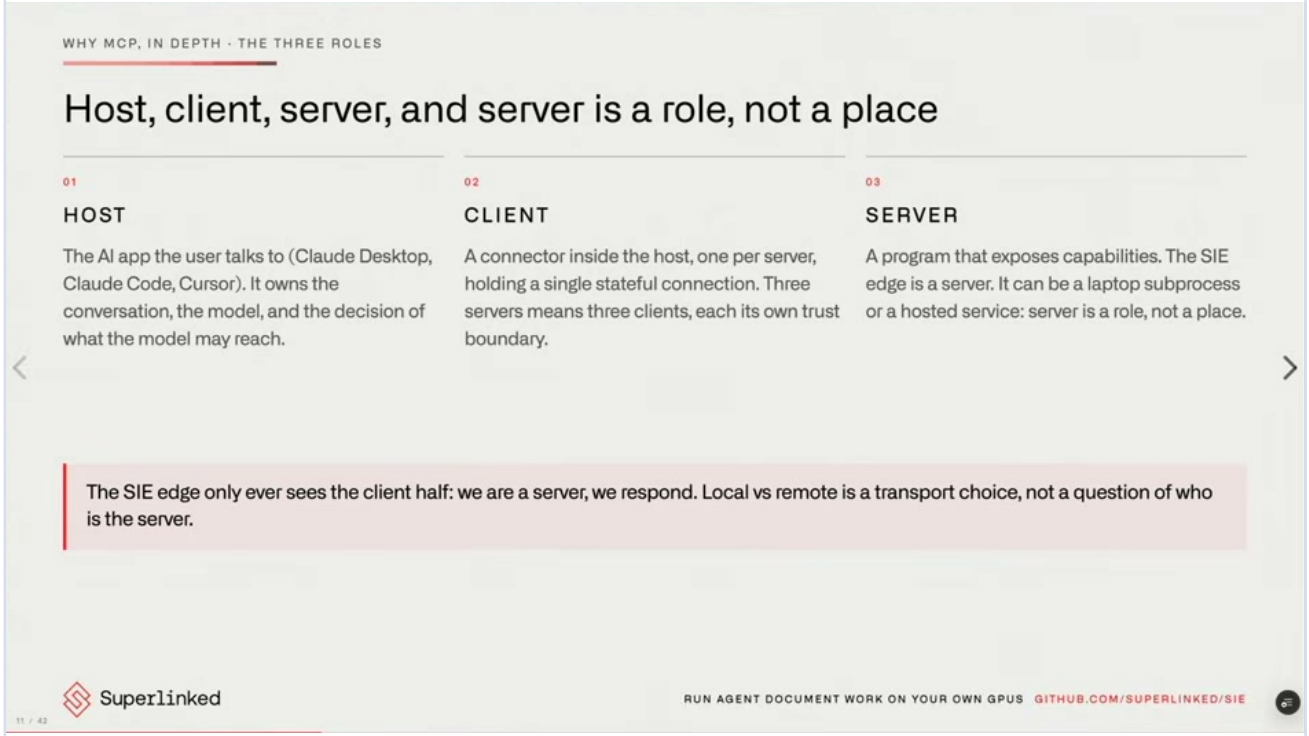
CORE IDEA

Standard boundaries reduce integration count and make capabilities portable across agents.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Host, client, and server are protocol roles

A server can be local or remote; deployment is a transport choice



NOTES

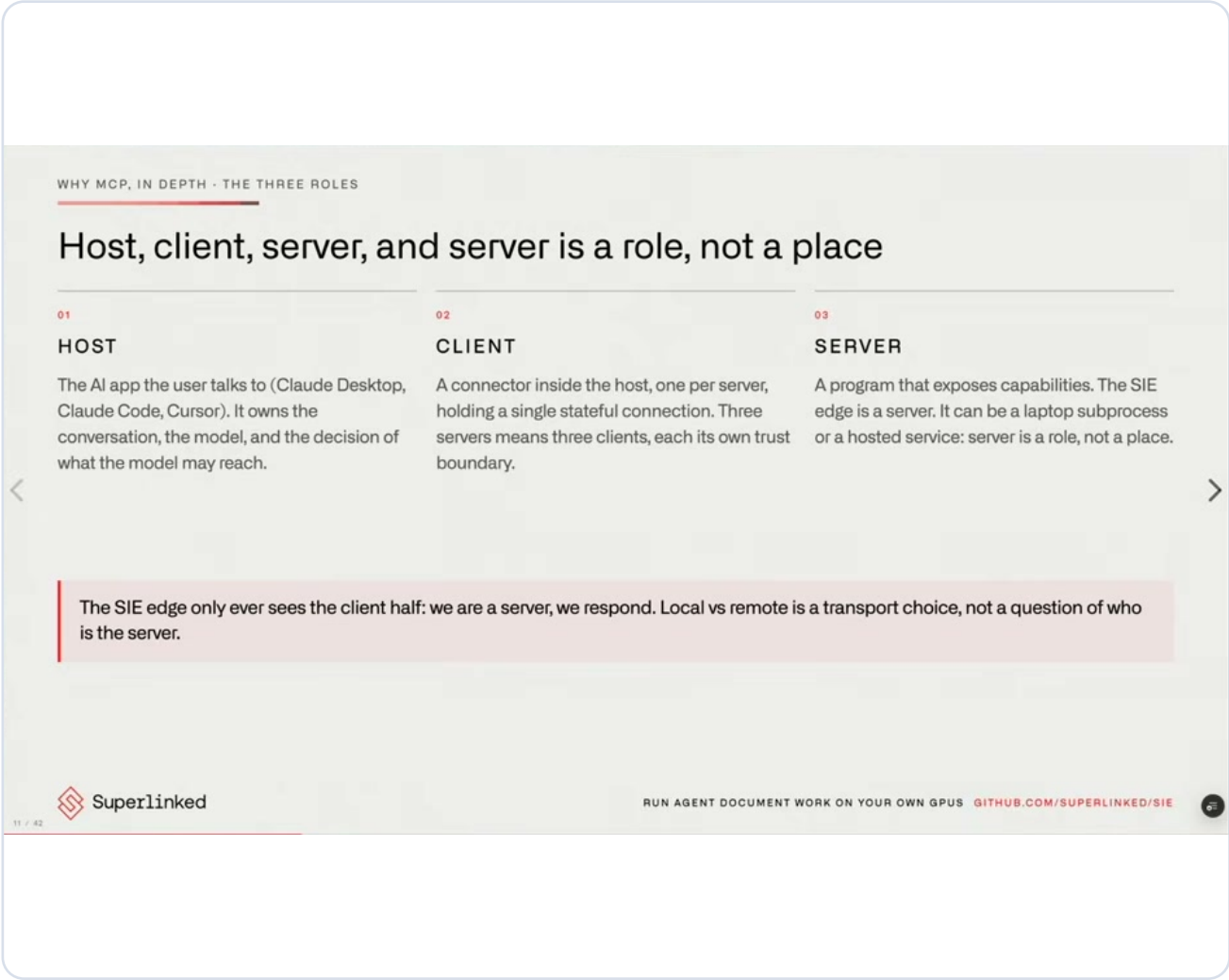
- The host owns the user-facing agent application and creates clients for external capability providers.
- The client maintains a connection, negotiates protocol features, and mediates requests and responses.
- The server exposes capabilities; the word server describes its protocol role rather than requiring a separate remote machine.

CORE IDEA

Protocol roles should not be confused with physical deployment locations.

JSON-RPC handshake establishes the contract

Initialization pins protocol version and advertised capabilities



NOTES

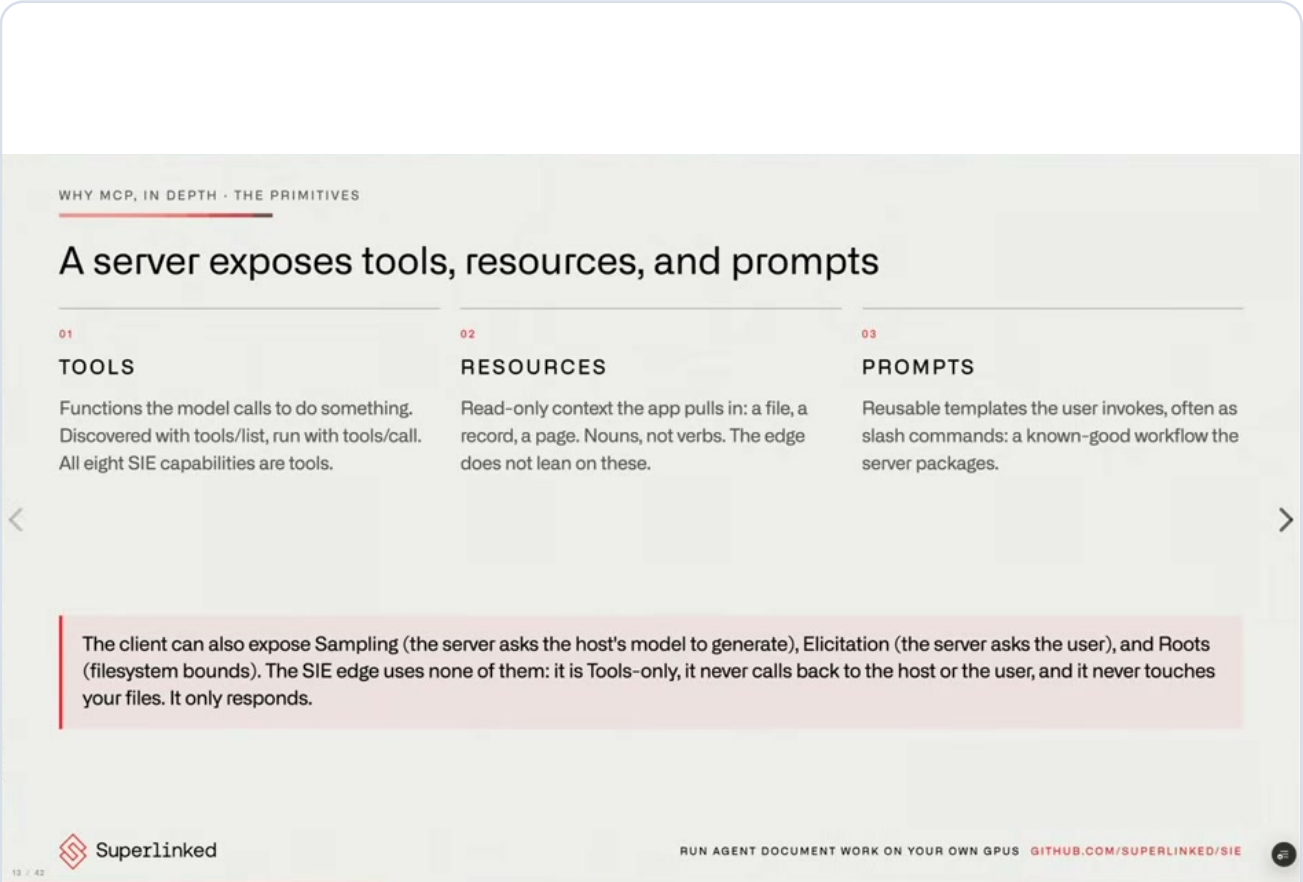
- MCP uses JSON-RPC 2.0 messages for requests, responses, and notifications between client and server.
- Initialization negotiates the protocol version and exchanges the capabilities each side supports.
- An initialized notification completes the handshake before normal tool discovery and invocation begin.

CORE IDEA

Explicit negotiation prevents clients from assuming unsupported behavior.

Servers expose tools, resources, and prompts

This privacy-oriented implementation deliberately keeps the surface narrow



NOTES

- Tools are callable actions, resources provide readable context, and prompts package reusable message templates.
- MCP clients can also expose facilities such as sampling, elicitation, and roots, enabling bidirectional patterns.
- For this privacy workflow, the edge primarily exposes server-side tools and avoids unnecessary client callbacks.

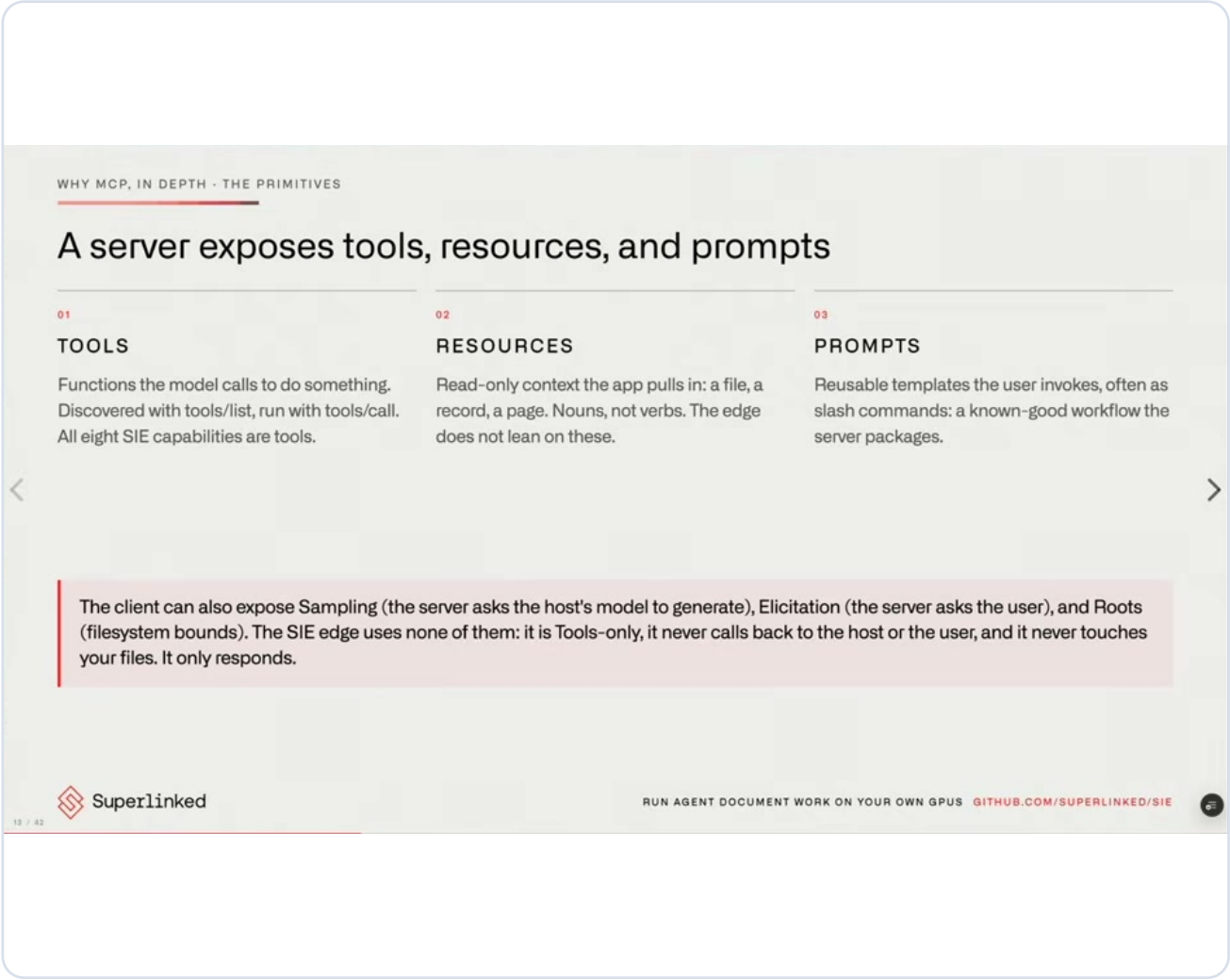
CORE IDEA

A smaller capability surface is easier to reason about and secure.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Capabilities are negotiated and discovered live

The tool list is runtime state, not hard-coded agent knowledge



NOTES

- After initialization, the client requests the server's current tools instead of relying on a fixed integration manifest.
- List-change notifications can reveal capabilities added or removed while the connection remains active.
- Dynamic discovery supports evolving tool bundles but requires clients to handle capability changes safely.

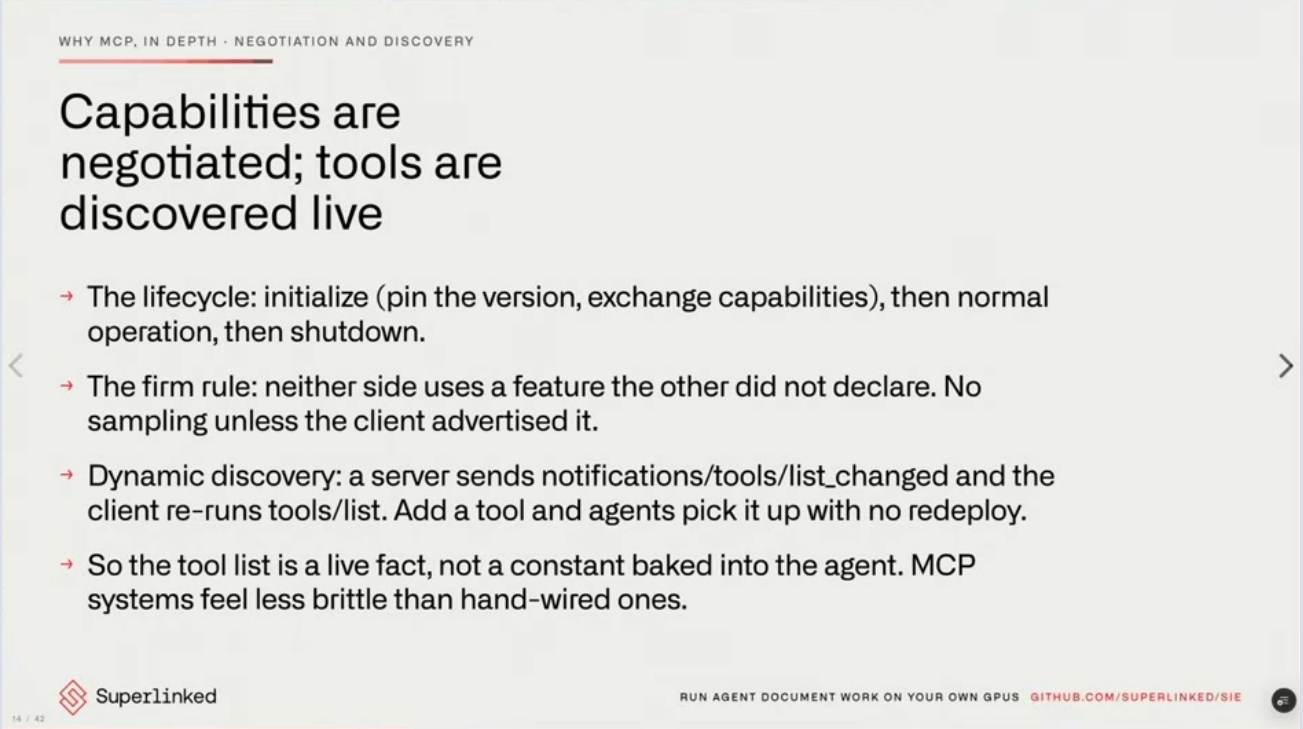
CORE IDEA

Live discovery reduces brittle wiring while preserving explicit capability checks.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Local and hosted MCP transports

Standard I/O suits child processes; Streamable HTTP suits hosted edges



NOTES

- With standard input/output, a host launches a local MCP server as a child process and exchanges messages directly.
- A hosted edge exposes an HTTP endpoint, accepts JSON-RPC requests, and can stream longer responses.
- The demonstrated private-cloud deployment uses the hosted transport so multiple clients can reach one controlled edge.

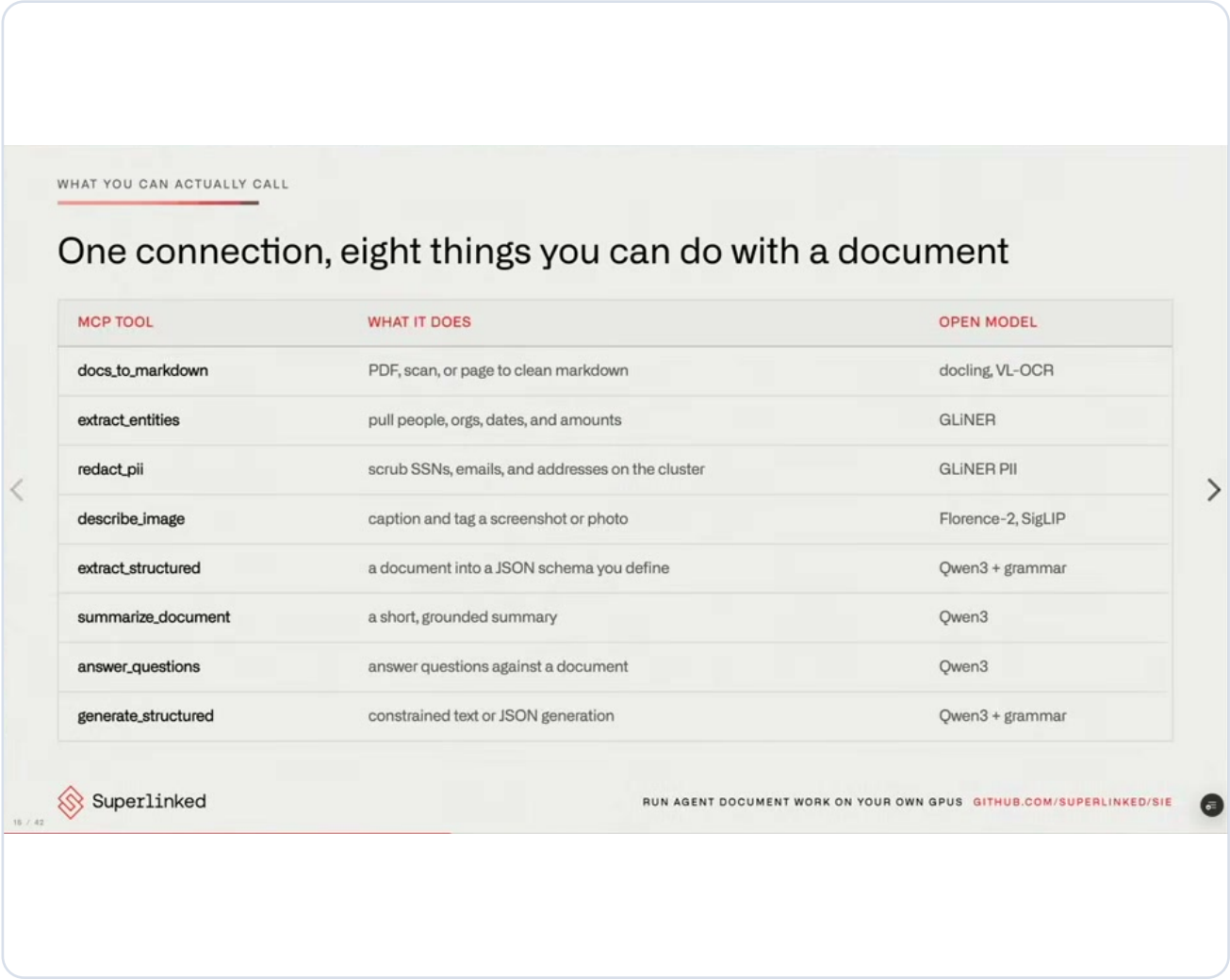
CORE IDEA

Choose transport according to deployment, not according to capability semantics.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

One connection exposes a document-tool bundle

Open models cover OCR, entities, images, structure, summaries, and questions



NOTES

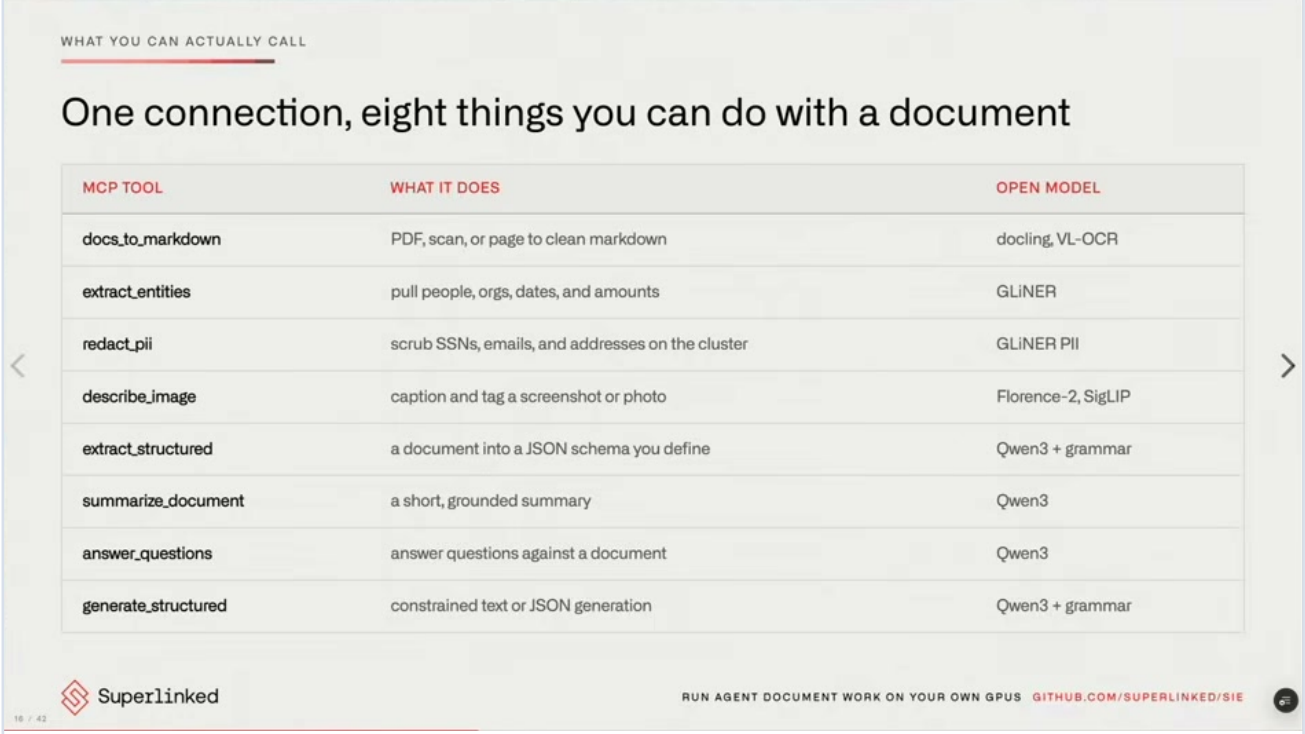
- The edge groups several document operations behind one MCP connection instead of requiring a service per model.
- Tasks include document-to-Markdown conversion, named-entity extraction, captioning, tagging, structured generation, summarization, and question answering.
- Model choices are replaceable and extensible; contributors can add tools while reusing the cluster's inference and scheduling layer.

CORE IDEA

The stable unit is the capability, not a permanently bound model endpoint.

Bring up the inference cluster

Use a managed key for quick testing or self-host the open stack



WHAT YOU CAN ACTUALLY CALL

One connection, eight things you can do with a document

MCP TOOL	WHAT IT DOES	OPEN MODEL
docs_to_markdown	PDF, scan, or page to clean markdown	docling, VL-OCR
extract_entities	pull people, orgs, dates, and amounts	GLiNER
redact_pii	scrub SSNs, emails, and addresses on the cluster	GLiNER PII
describe_image	caption and tag a screenshot or photo	Florence-2, SigLIP
extract_structured	a document into a JSON schema you define	Qwen3 + grammar
summarize_document	a short, grounded summary	Qwen3
answer_questions	answer questions against a document	Qwen3
generate_structured	constrained text or JSON generation	Qwen3 + grammar

Superlinked

RUN AGENT DOCUMENT WORK ON YOUR OWN GPUS GITHUB.COM/SUPERLINKED/SIE

NOTES

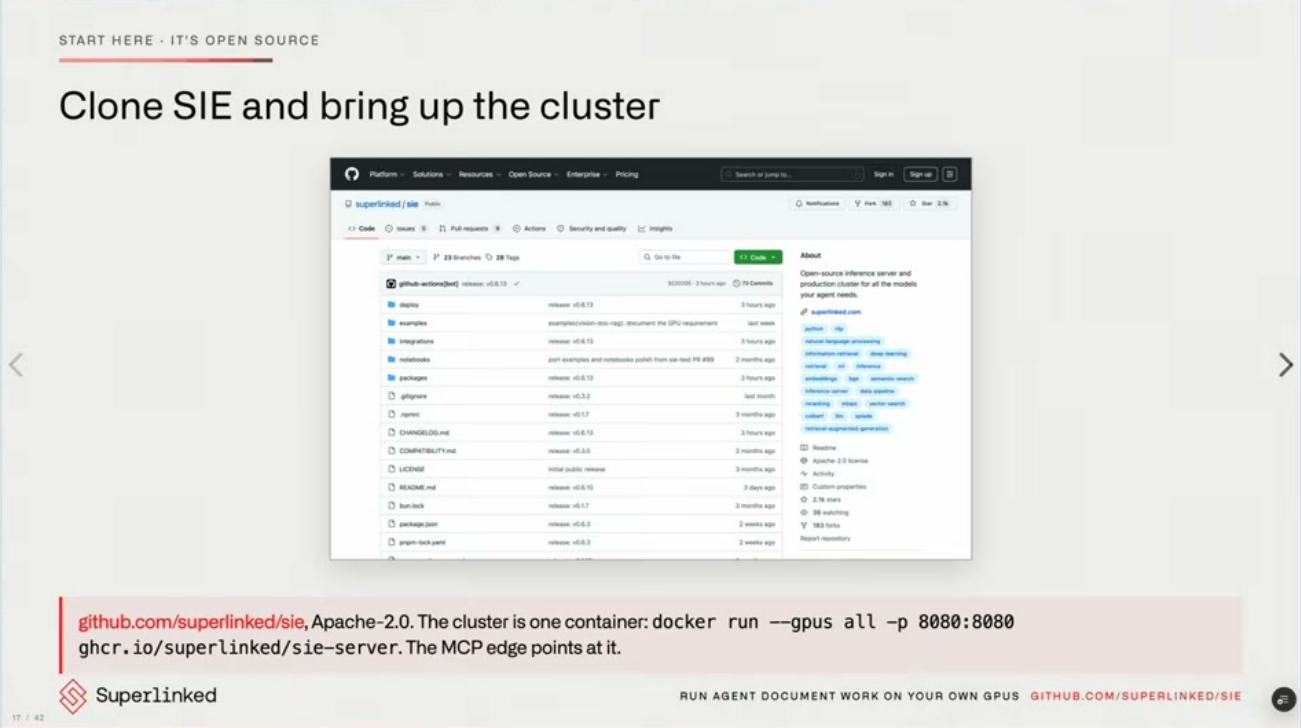
- The setup begins by cloning the open-source inference repository and starting the cluster components.
- A managed cluster can shorten evaluation time, while local containers or self-hosted infrastructure preserve full control.
- For production, the cluster can be deployed through the project's Kubernetes, Helm, and Terraform assets.

CORE IDEA

The managed demo path is optional; the architecture can remain fully self-hosted.

Point the MCP edge at the cluster

The edge needs an endpoint and credential, not access inside the agent



NOTES

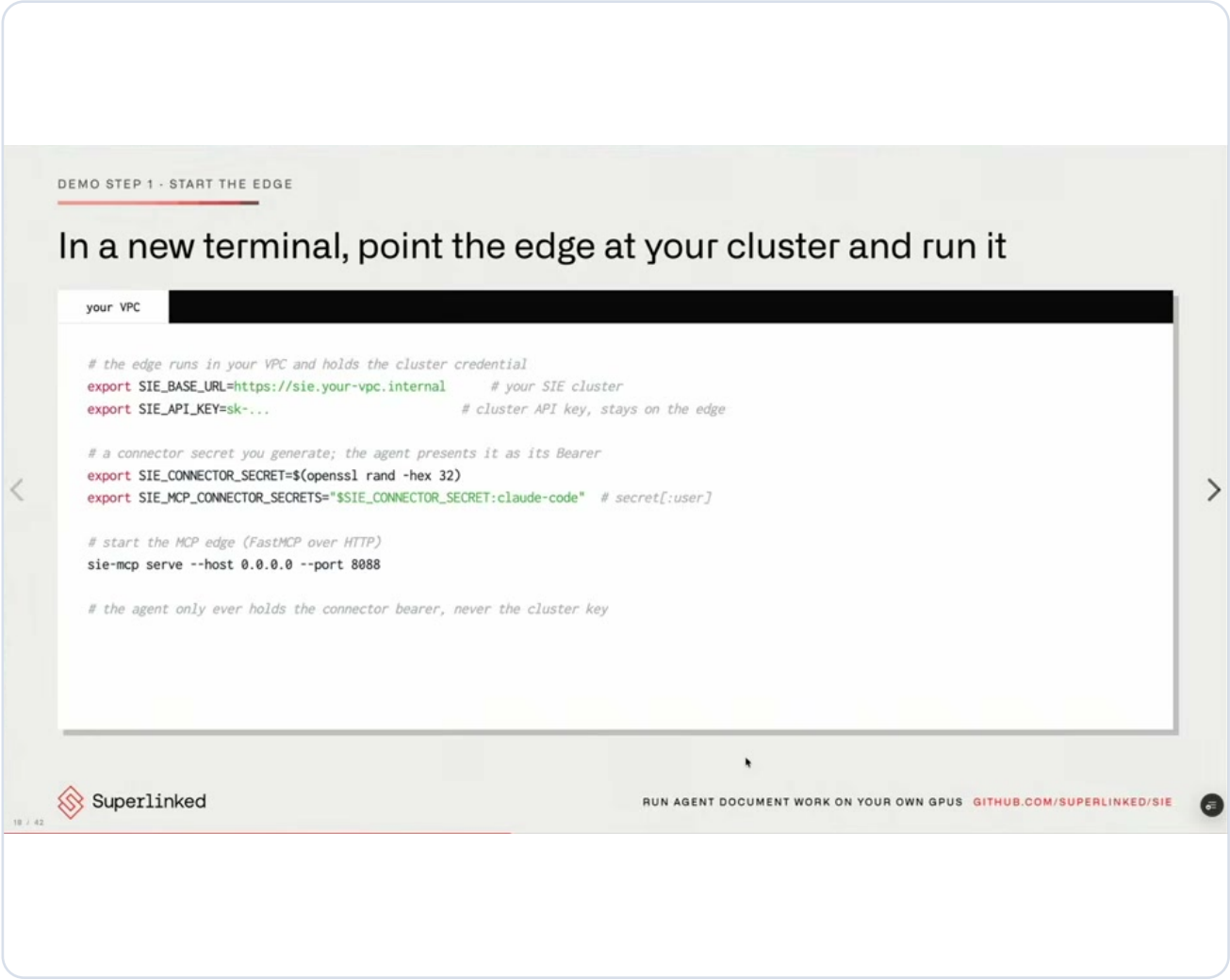
- A separate terminal configures the edge with the inference endpoint and API key, then starts the hosted service.
- Credentials remain at the edge boundary rather than entering the agent's prompt or local plugin state.
- The same pattern works with a provider-managed cluster or a cluster operated entirely by the customer.

CORE IDEA

Keep infrastructure credentials where requests are translated into controlled jobs.

Connect the agent with two commands

Registration is small because the complex work stays behind MCP



NOTES

- The client registers the hosted MCP endpoint and then discovers the document capabilities exposed by that server.
- Minimal client configuration avoids duplicating model runtime, queueing, and document-processing logic in each agent.
- Operational simplicity on the client side depends on maintaining the edge and cluster correctly behind the interface.

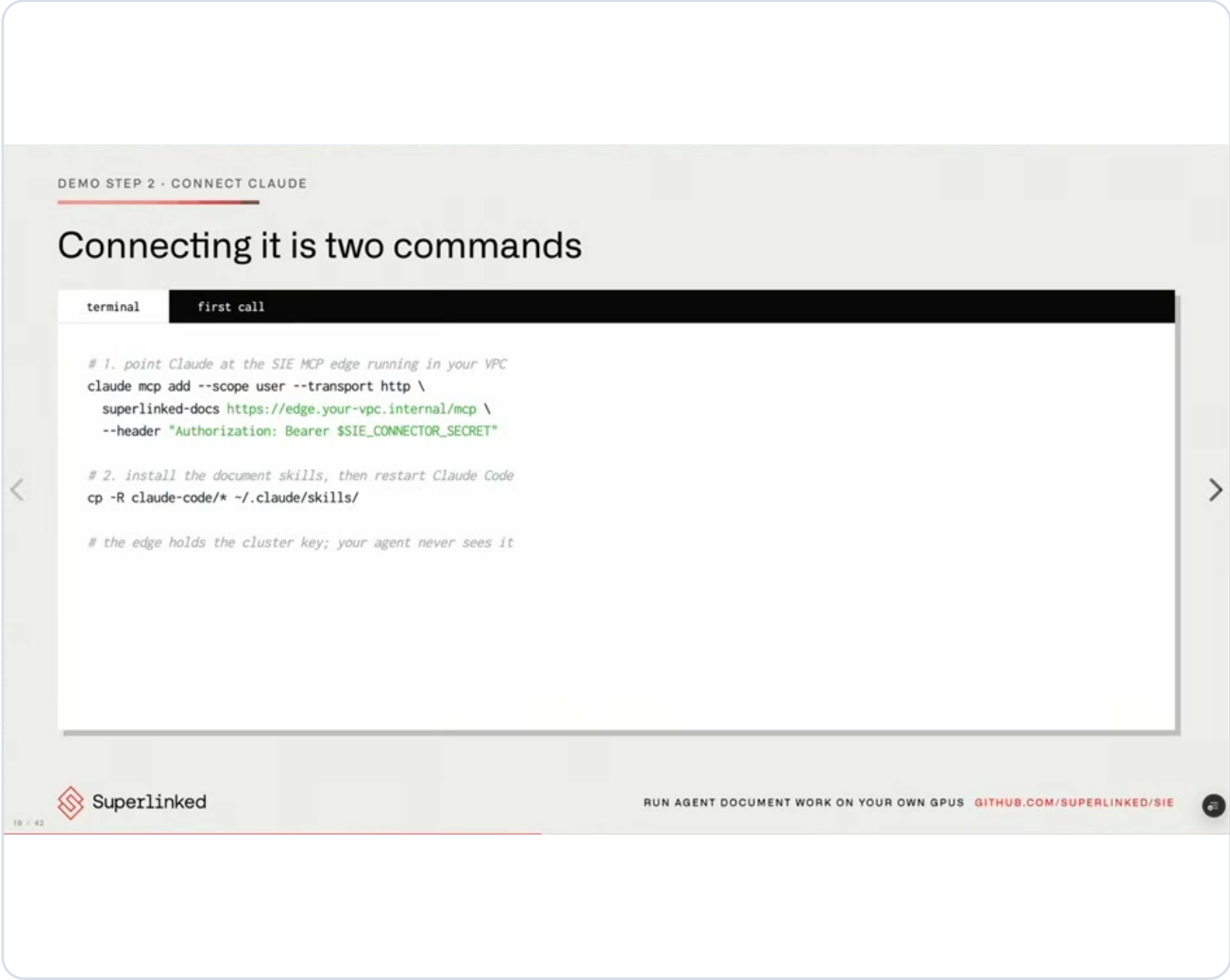
CORE IDEA

A thin client preserves the normal agent workflow while centralizing complexity.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Contracts move through parse, redact, and reveal

The end-to-end workflow creates a private knowledge graph



NOTES

- Synthetic contracts enter an inbox and are parsed into linked facts suitable for an Obsidian knowledge graph.
- Sensitive spans are identified and replaced before downstream agent use, while the protected originals remain inside the cluster.
- Controlled reveal or retrieval can return only the fields and relationships needed for a specific task.

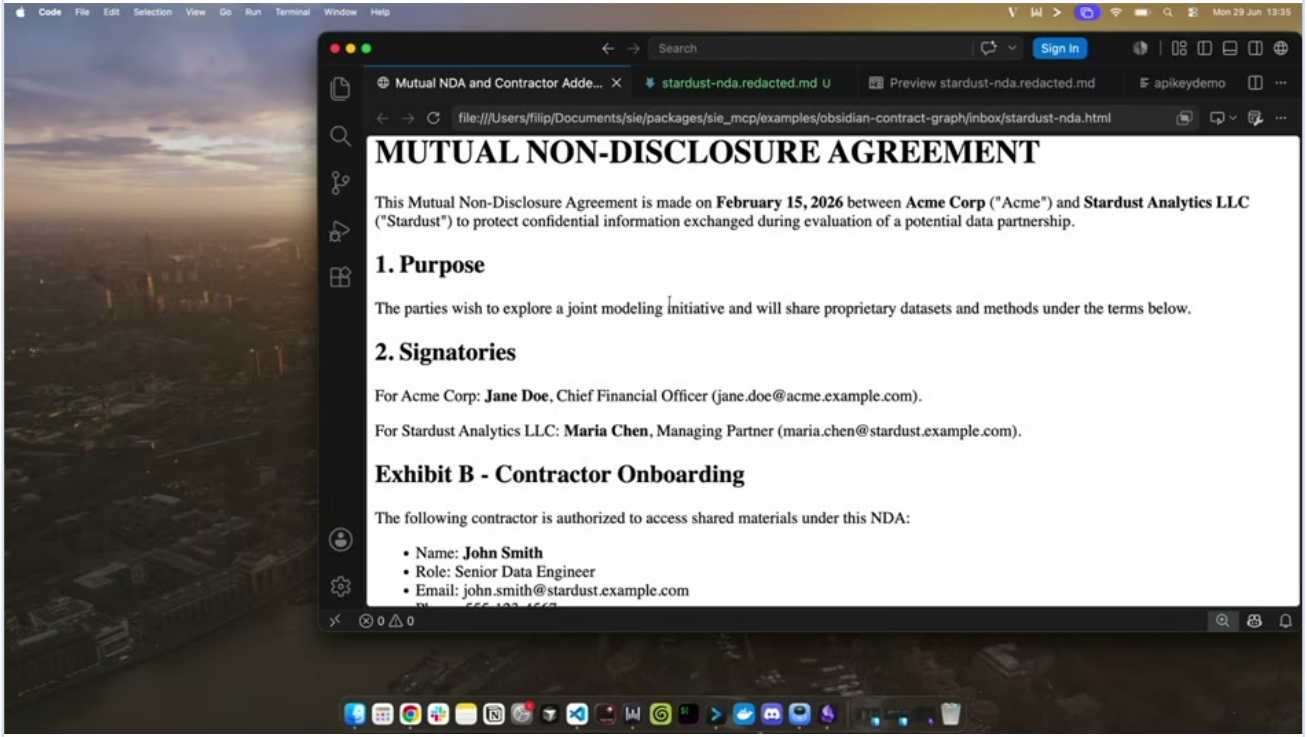
CORE IDEA

Document intelligence becomes useful without exposing the complete source corpus.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

The original NDA contains sensitive fields

The demo uses synthetic data, not the records that inspired the project



NOTES

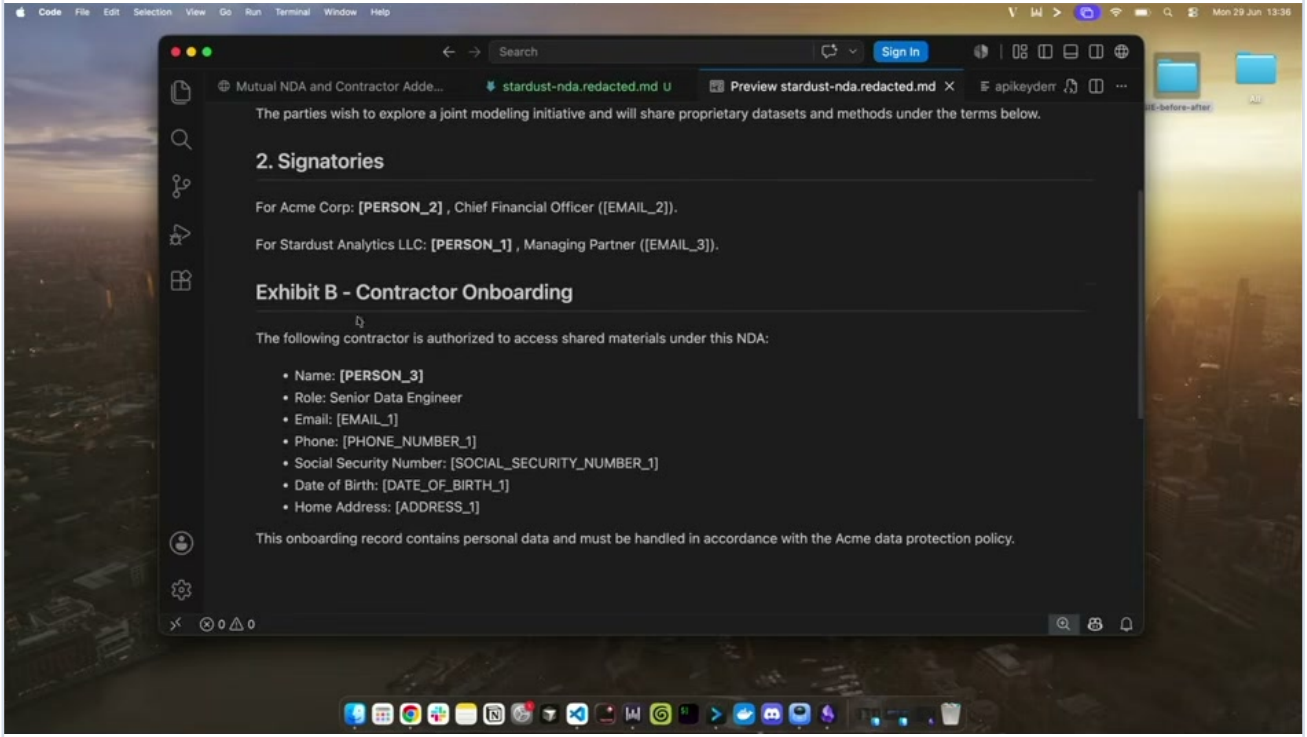
- The mutual non-disclosure agreement contains names, contact details, signatory information, and contractor onboarding data.
- A natural-language request asks the coding agent to redact the NDA rather than manually choosing a model or script.
- The source remains available to the private worker even though the external agent should receive only the sanitized result.

CORE IDEA

Natural-language convenience does not require giving the agent the unredacted document.

The redacted Markdown preserves structure

Entities are masked while useful headings and relationships remain



NOTES

- The output retains sections such as purpose, signatories, and contractor onboarding so the document remains usable.
- Detected person, email, phone, and similar spans are replaced with labels or redaction tokens.
- The run reports label counts and affected spans; these are model outputs and still require validation for missed or false detections.

CORE IDEA

Good redaction removes identifiers without destroying the document's operational meaning.

Scanned agreements require optical character recognition

An image-only contract cannot be treated as ordinary text



SERVICES AGREEMENT - EXHIBIT C

This onboarding record is provided to Umbrella Logistics Inc. under the Master Services Agreement dated April 9, 2026.

Contractor Details

Name: Priya Nair
Role: Field Operations Lead
Email: priya.nair@umbrella-logistics.example
Phone: +1 (415) 555-0148
Social Security No.: 482-19-7731
Date of Birth: 1989-03-22
Home Address: 77 Maple Court, Portland, OR 97204

Total contract value: $1,240,000 over 24 months.
This page is a scanned facsimile of the signed original.

NOTES

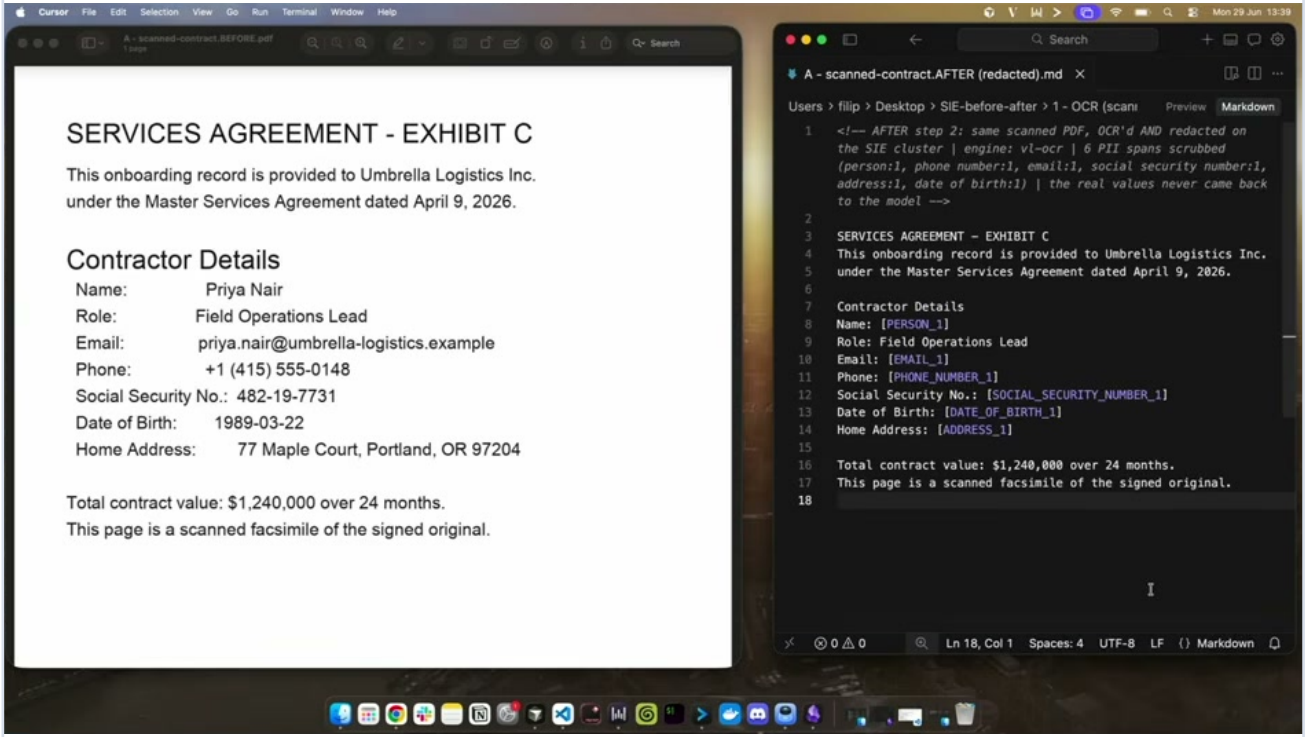
- The services agreement is shown as a scan containing contractor details and compensation information.
- Because the page has no reliable text layer, an optical character recognition model must first recover its content.
- Open OCR models are presented as suitable for this stage, but real quality varies with scan condition, language, and layout.

CORE IDEA

Scanning changes the task from text parsing into multimodal document processing.

OCR and redaction run inside the private boundary

The result is Markdown with sensitive content removed



NOTES

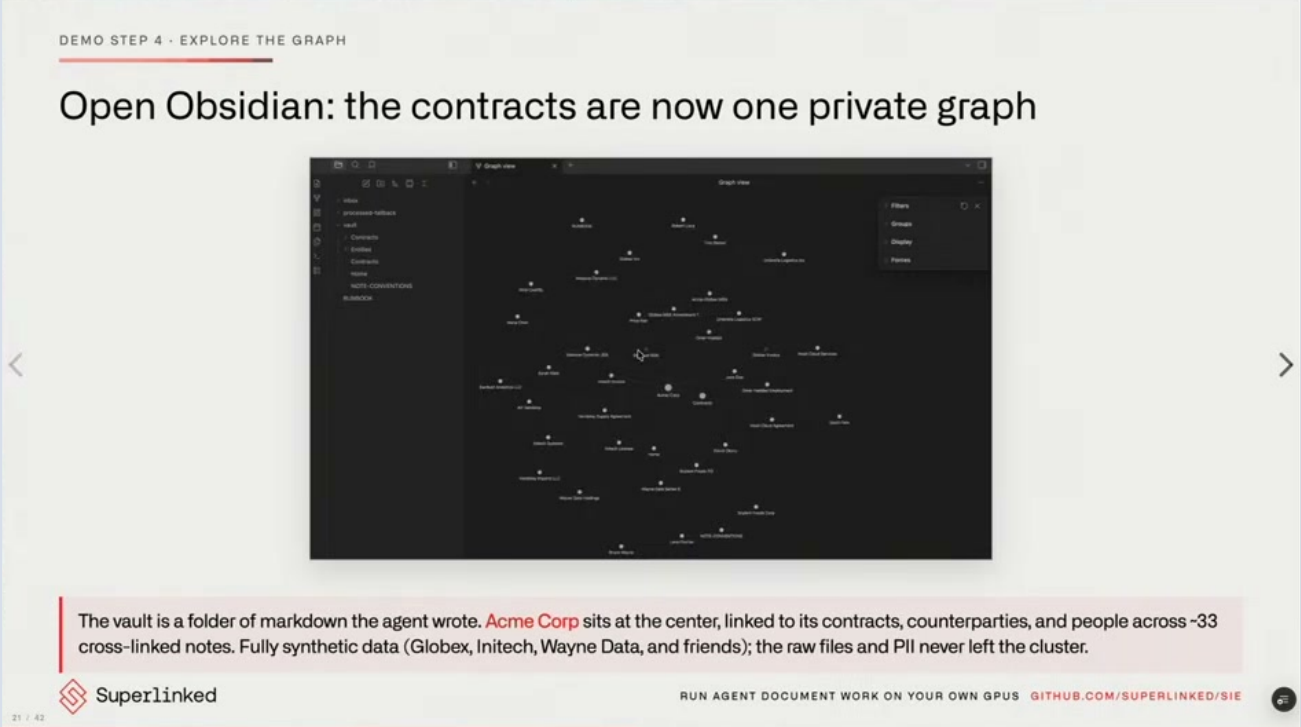
- The private worker converts the scanned agreement into structured Markdown and then applies entity detection and redaction.
- The terminal shows processing and structured output while the source page remains visible for comparison.
- A token-savings indicator is part of the prototype, but the talk notes that its accounting still needs refinement.

CORE IDEA

The expensive multimodal stage can stay private and return compact text.

The sensitive identifier never leaves the cluster

Only the sanitized artifact crosses back to the external agent



NOTES

- The core security claim is that raw identifiers are processed inside the controlled cluster rather than transmitted to the outside model.
- The agent receives a redacted representation sufficient for further reasoning, coding, or knowledge-graph updates.
- This is a demonstrated architecture claim, not a formal proof; the Q&A; acknowledges that additional guardrails and leakage testing are valuable.

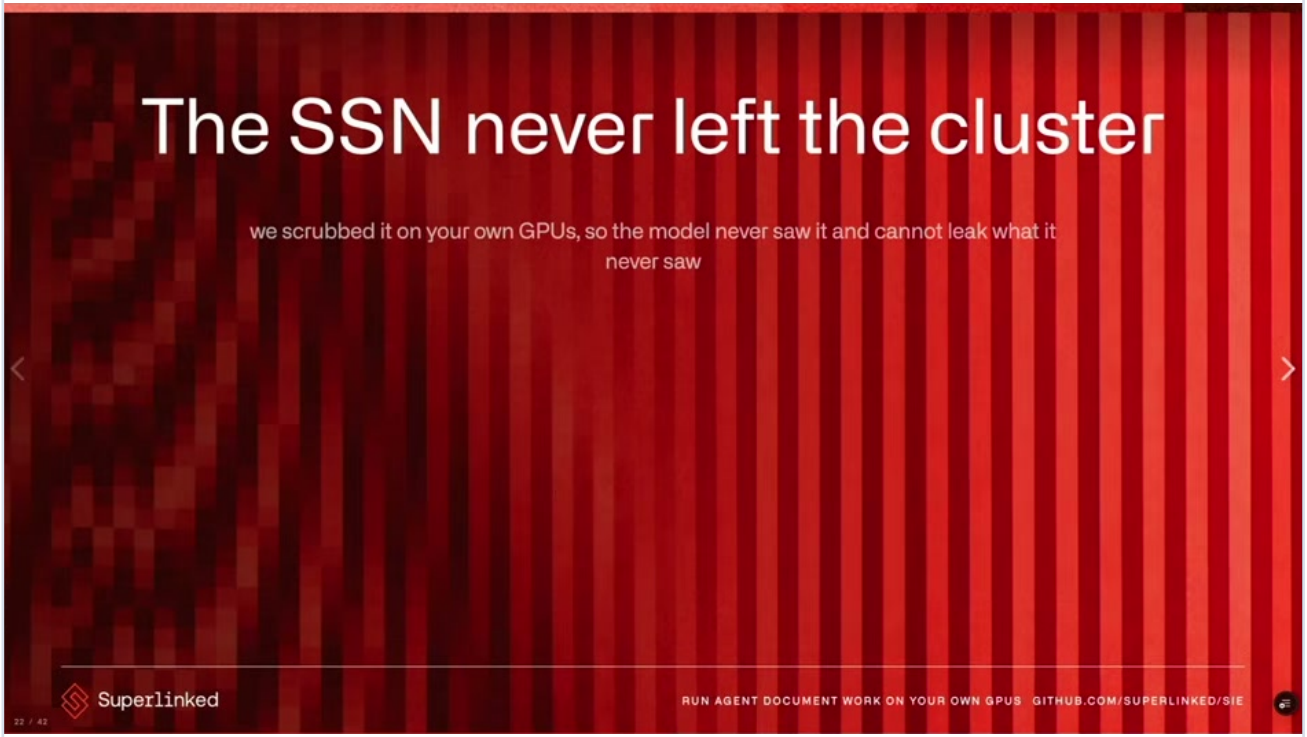
CORE IDEA

Privacy depends on enforcing the boundary, not merely promising local storage.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

A self-hosted inference cluster supplies the backbone

Stateless gateway, pooled work streams, and GPU workers



NOTES

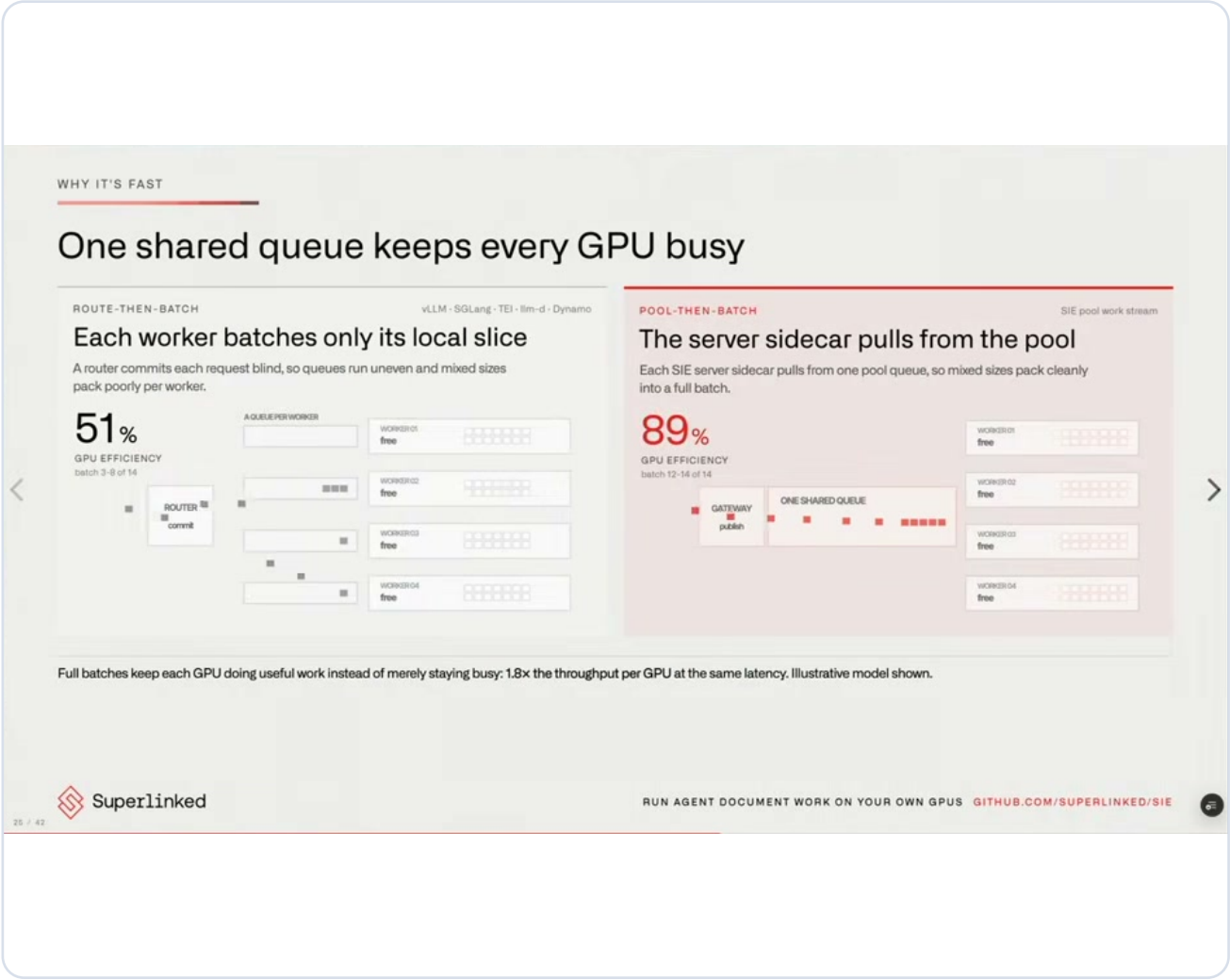
- A stateless gateway accepts jobs and directs them toward pools of workers suited to different model families and hardware.
- Worker pods keep GPUs supplied with batched tasks while preserving independent scaling and failure domains.
- The open infrastructure can run in the customer's cloud, subject to the customer's hardware, operations, and security controls.

CORE IDEA

The agent interface is simple because scheduling and serving are solved underneath it.

Shared work queues improve GPU utilization

Per-worker batching outperformed fragmented local queues in the reported tests



NOTES

- Fragmented queues can leave one GPU idle while another worker holds pending work, reducing end-to-end throughput.
- A shared pool lets sidecars pull available jobs and batch work at the worker, keeping accelerators busier.
- The utilization figures are reported demo results and are not independently benchmarked across every workload or cluster shape.

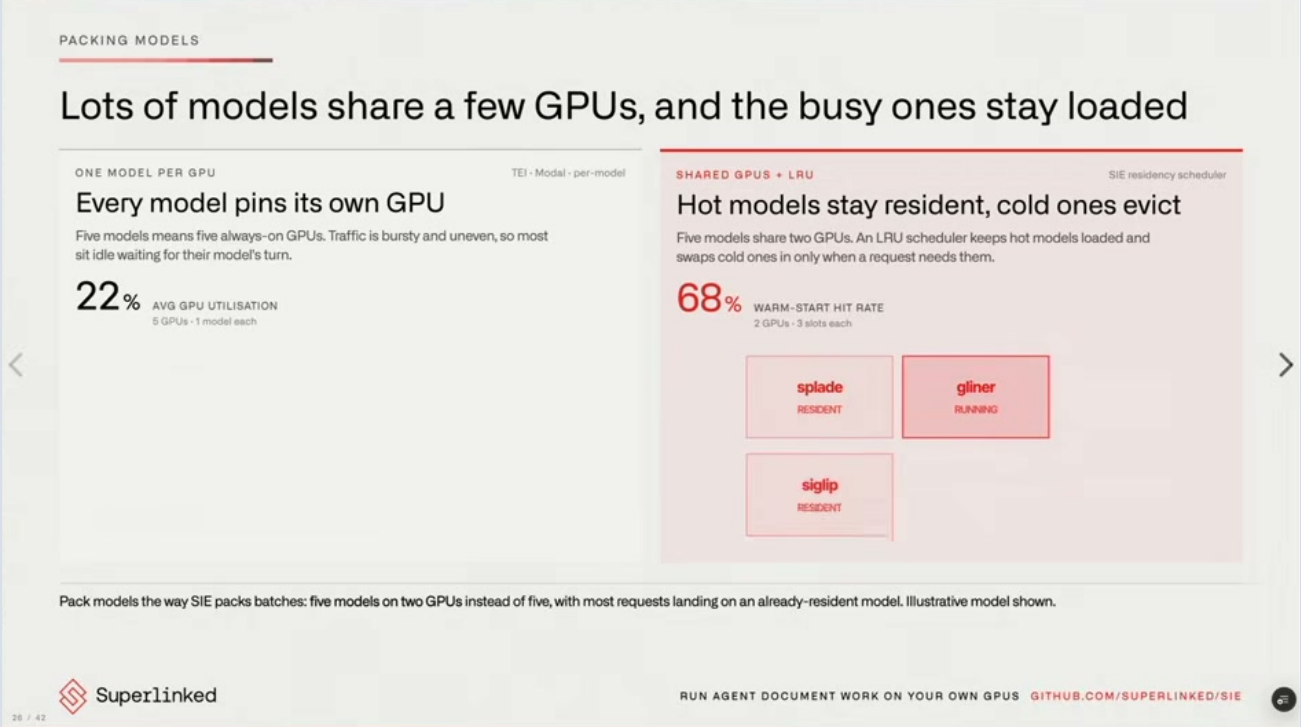
CORE IDEA

Queue architecture can matter as much as single-model kernel speed.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Hot models stay resident while cold models evict

Many small task models can share a limited GPU pool



NOTES

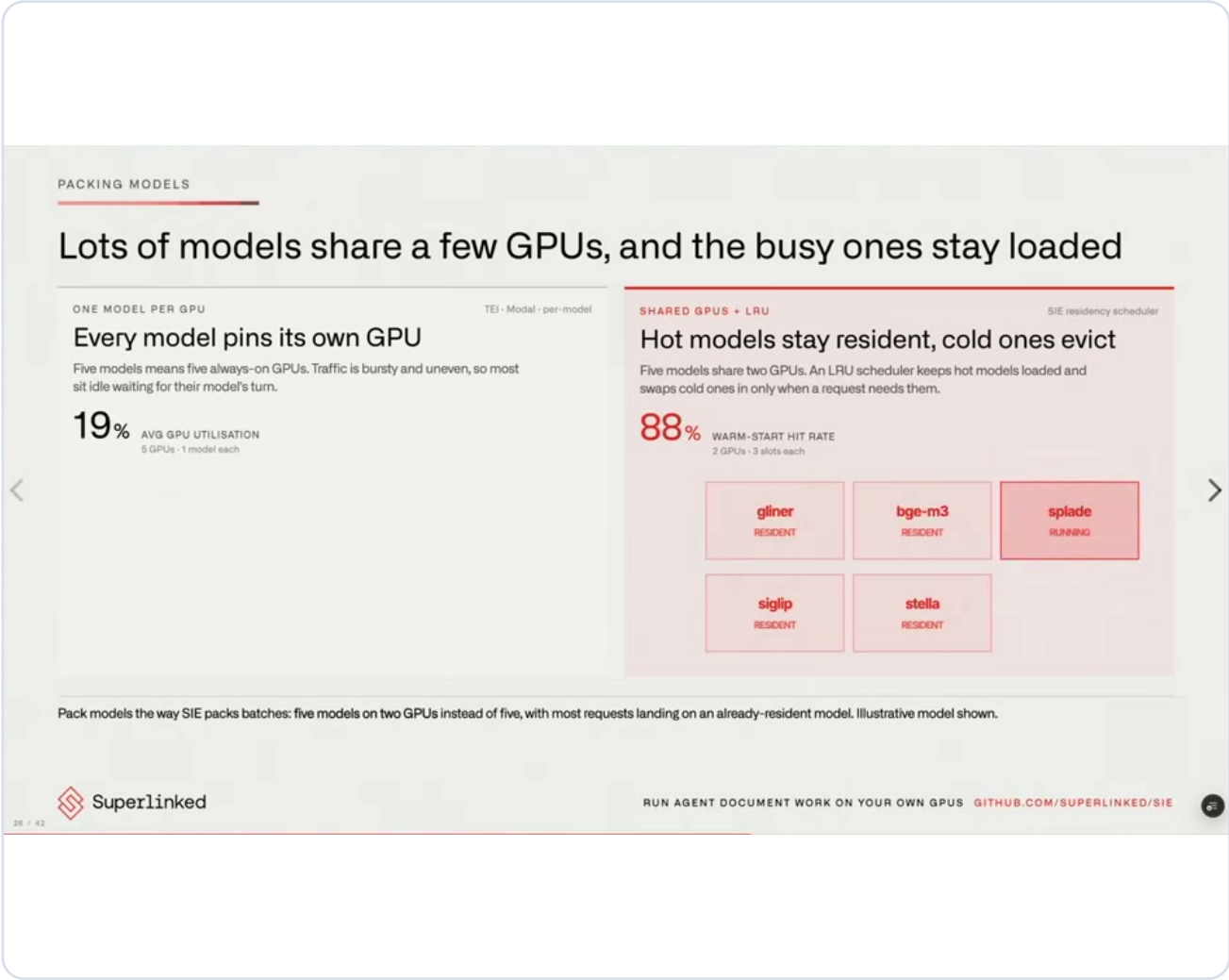
- Pinning every model to a dedicated GPU wastes memory and hardware when demand is uneven.
- Frequently used models remain loaded, while colder checkpoints can be removed and restored when needed.
- This is especially useful for small named-entity, embedding, captioning, or generation models and multiple task-specific LoRA adapters.

CORE IDEA

Dynamic residency turns scarce GPU memory into a shared model cache.

One cluster can select models per job

Capabilities remain stable even as underlying model choices change



NOTES

- Document parsing, embeddings, image captioning, code generation, and policy checks have different quality and latency needs.
- The cluster can map each job to a compatible model and worker pool rather than exposing a fixed model endpoint to the agent.
- Selection can incorporate evaluation quality, cost, hardware type, and whether a model is already warm.

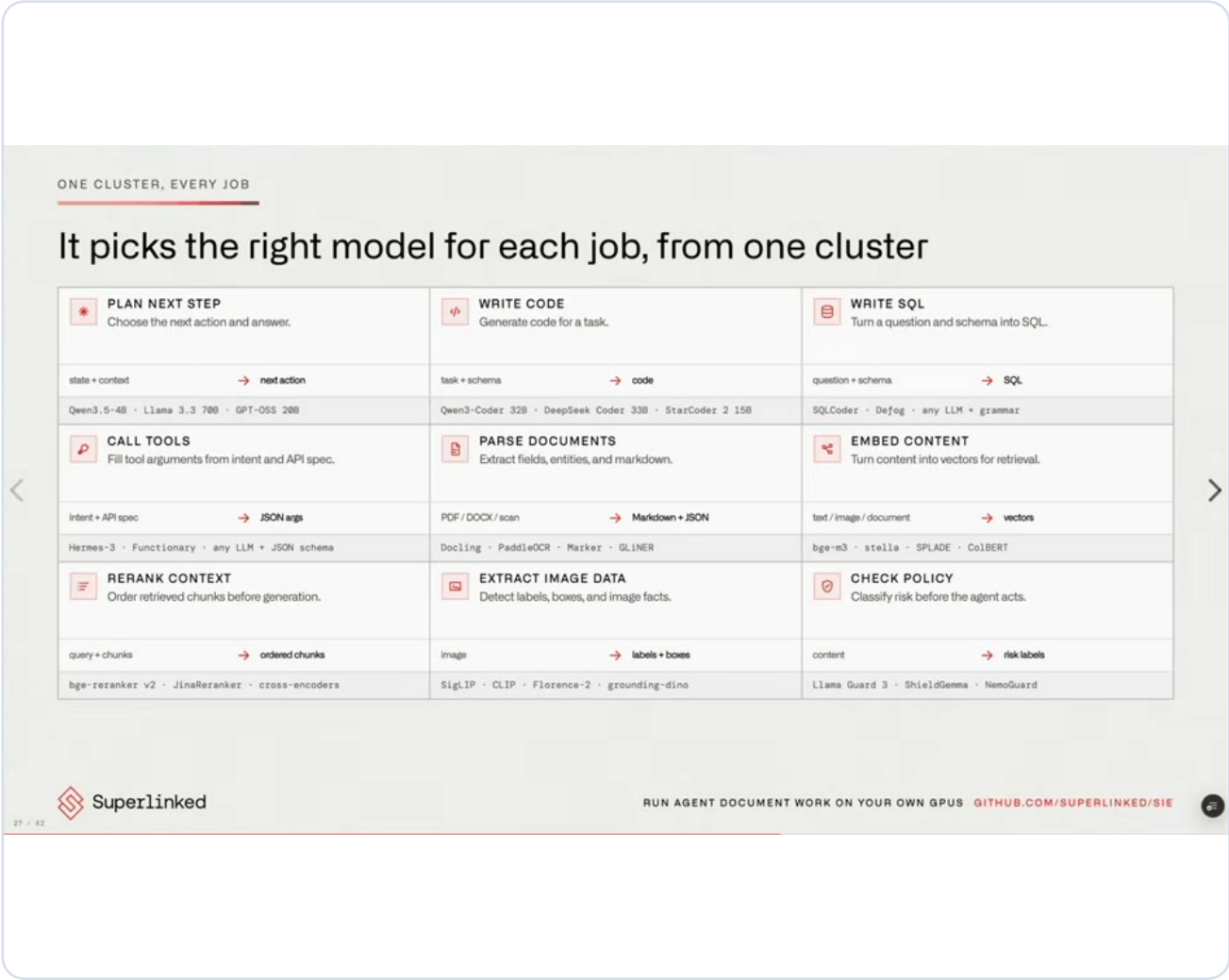
CORE IDEA

Route by task requirements instead of forcing one model across every capability.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Reported embedding economics

Near-frontier quality at substantially lower cost in selected MTEB tests



NOTES

- The talk reports its best embedding model reaching 99.8 percent of the frontier reference on selected Massive Text Embedding Benchmark tasks.
- It also reports roughly a twelvefold cost advantage from owning and operating the inference path rather than paying a metered API price.
- These figures apply to the presented evaluation setup; they should not be generalized without workload-specific replication.

CORE IDEA

Evaluate end-to-end cost and quality on the tasks that production will actually run.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Cheap LoRA adaptation can improve niche retrieval

Reported legal-domain gain from a low-cost fine-tune



NOTES

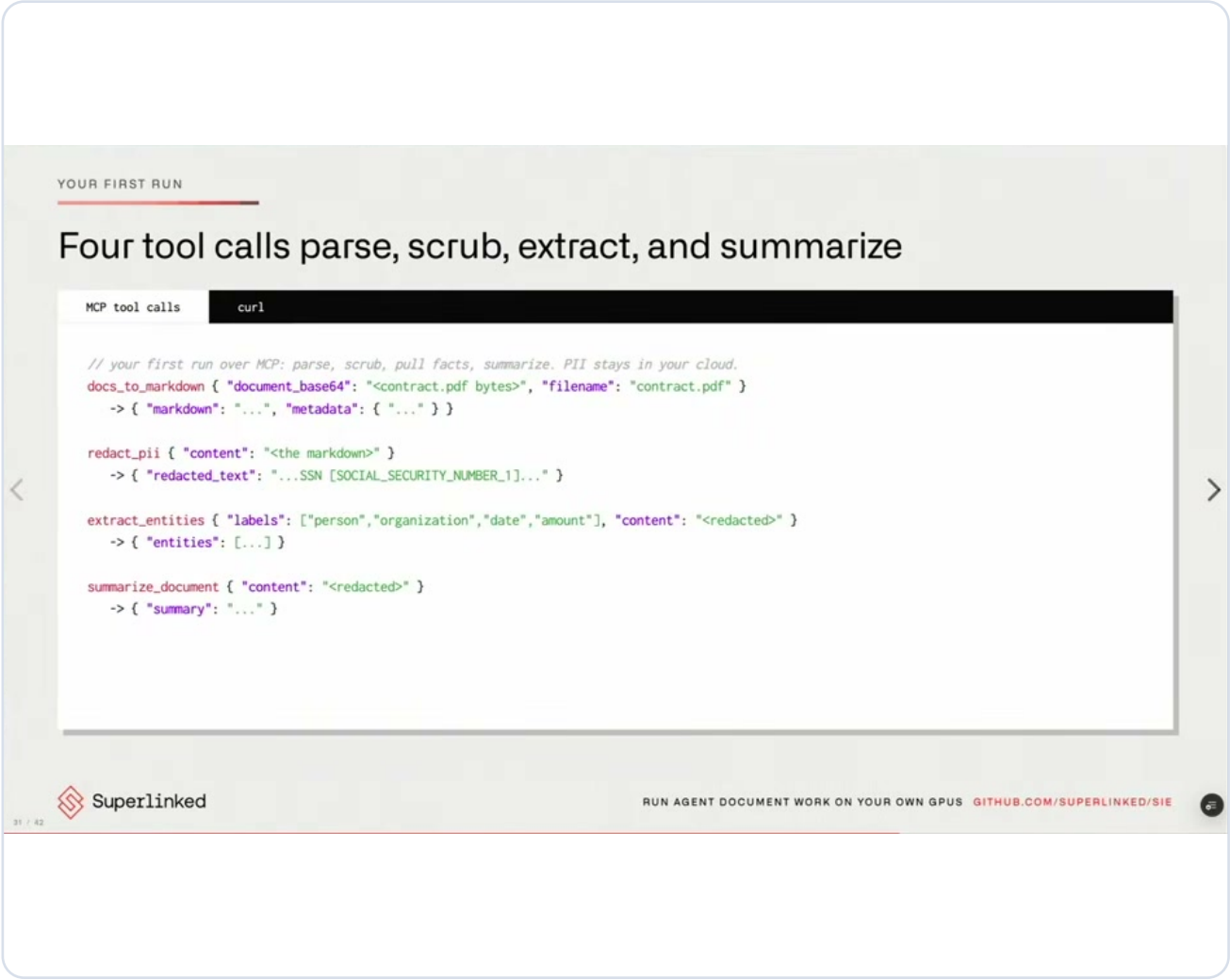
- Low-rank adaptation changes a small set of parameters, making domain specialization cheaper than full model fine-tuning.
- The slide reports a $0.80 LoRA producing an eighteen-percent legal retrieval lift for a narrow experiment, including German legal language work.
- A niche gain does not imply the same improvement elsewhere; held-out task evaluation is required before deployment.

CORE IDEA

Self-hosting makes rapid, domain-specific model experiments operationally practical.

Four tool calls compose the document workflow

Parse, scrub, extract, and summarize remain separate capabilities



NOTES

- The orchestration layer chains small tools rather than hiding every transformation inside one opaque prompt.
- Parsing creates a normalized artifact, scrubbing removes sensitive spans, extraction returns facts, and summarization produces a compact narrative.
- Separate stages make intermediate outputs inspectable and allow models to be replaced or evaluated independently.

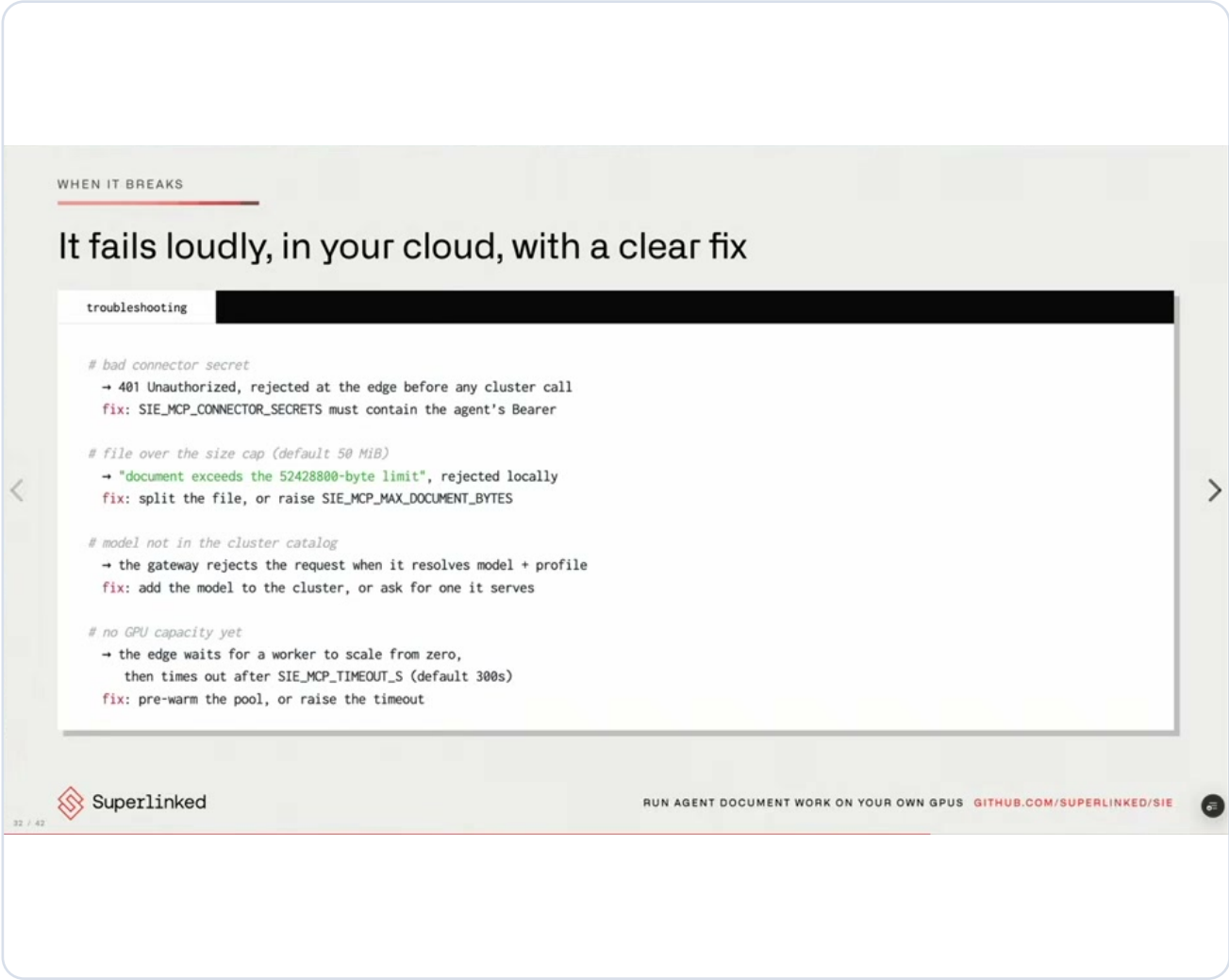
CORE IDEA

Composable tools create clearer checkpoints for privacy, quality, and debugging.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Failures should be loud and actionable

Diagnostics stay inside the controlled environment



NOTES

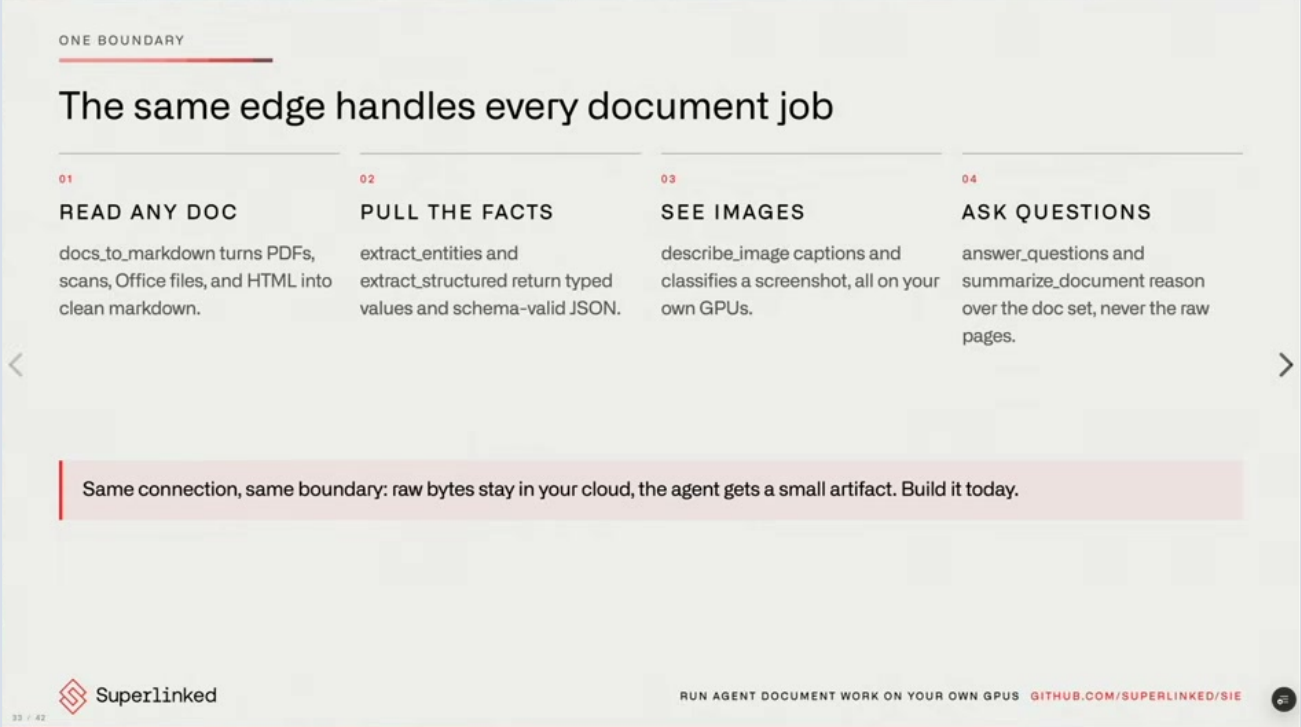
- The edge reports a clear failure and remedy instead of returning an apparently successful but incomplete artifact.
- Operational logs remain in the customer's cloud, where engineers can inspect model, parser, queue, and credential issues.
- Explicit errors are essential for sensitive workflows because silent partial processing can leave identifiers unredacted.

CORE IDEA

A privacy pipeline must fail closed and explain how to recover.

The same edge handles every document job

Reading, extraction, visual understanding, and questions share one interface



NOTES

- One MCP edge can read supported documents, extract facts, inspect embedded images, and answer targeted questions.
- Each capability may use a different model or pipeline, but the agent sees a consistent discovery and invocation pattern.
- Centralization simplifies clients while making the edge a critical component that requires authentication, policy, and observability.

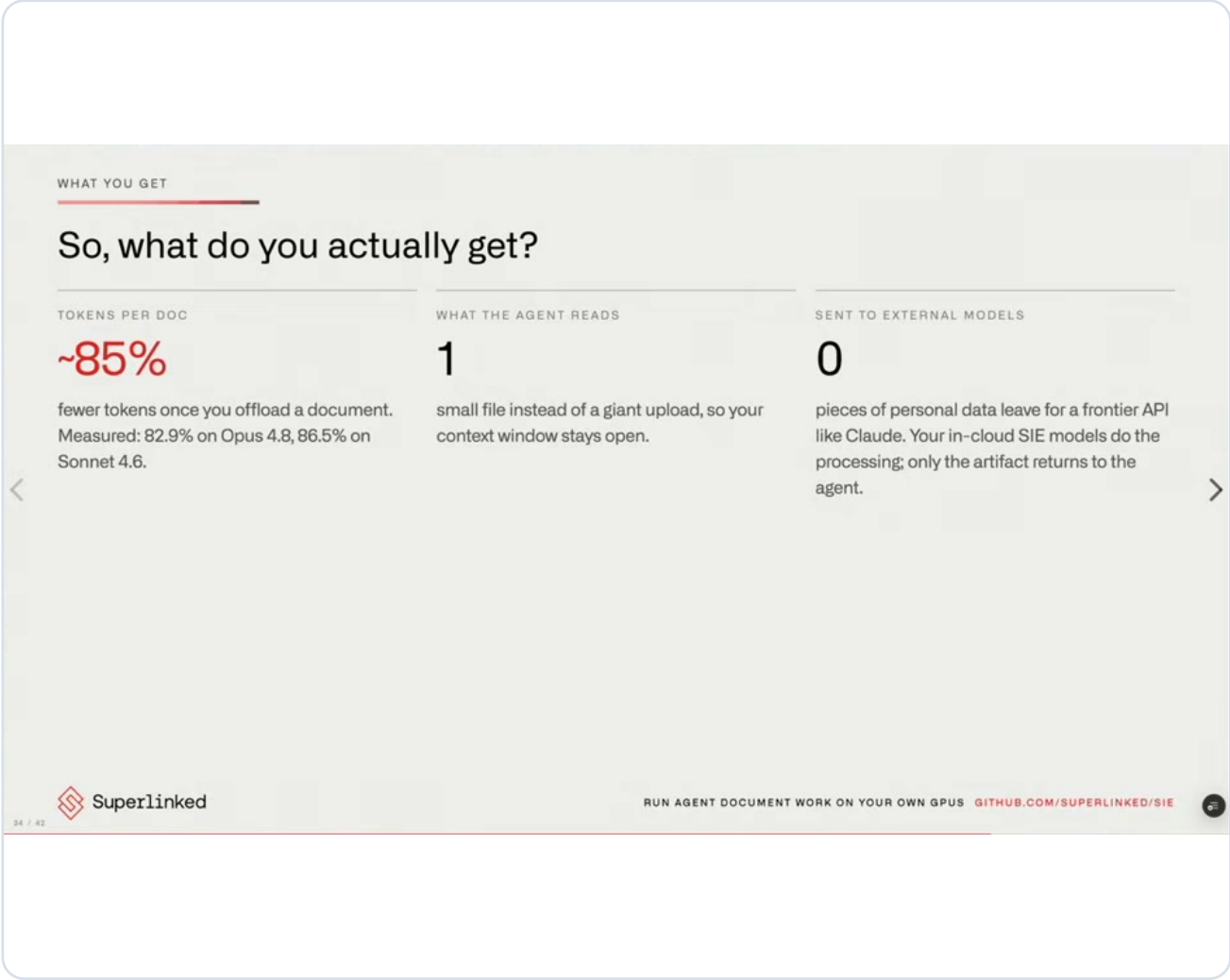
CORE IDEA

A unified interface should not erase the controls needed behind each capability.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Operational outcomes claimed by the prototype

Large artifact reduction, one edge, and no raw-data exposure



NOTES

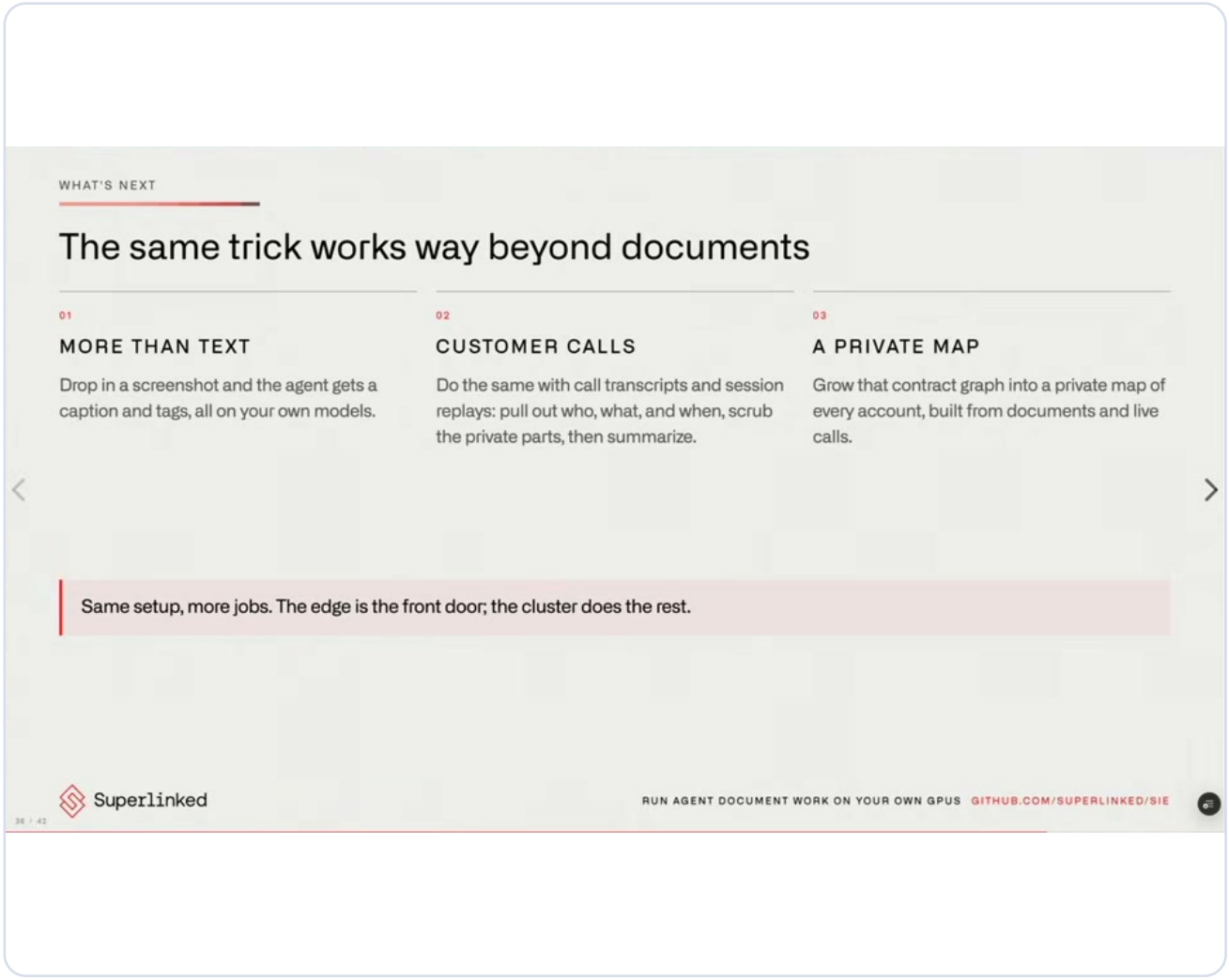
- The completed slide summarizes the prototype with an eighty-five-percent reduction, one reusable edge, and zero raw sensitive data sent outward.
- The reduction reflects the presented artifact-sizing example rather than a universal result for every document workload.
- The zero-exposure goal assumes the edge, cluster, tools, and client requests preserve the intended boundary.

CORE IDEA

Treat headline metrics as hypotheses to validate against real documents and threat models.

The pattern extends beyond documents

Calls, transcripts, graphs, and structured generation can use the same boundary



NOTES

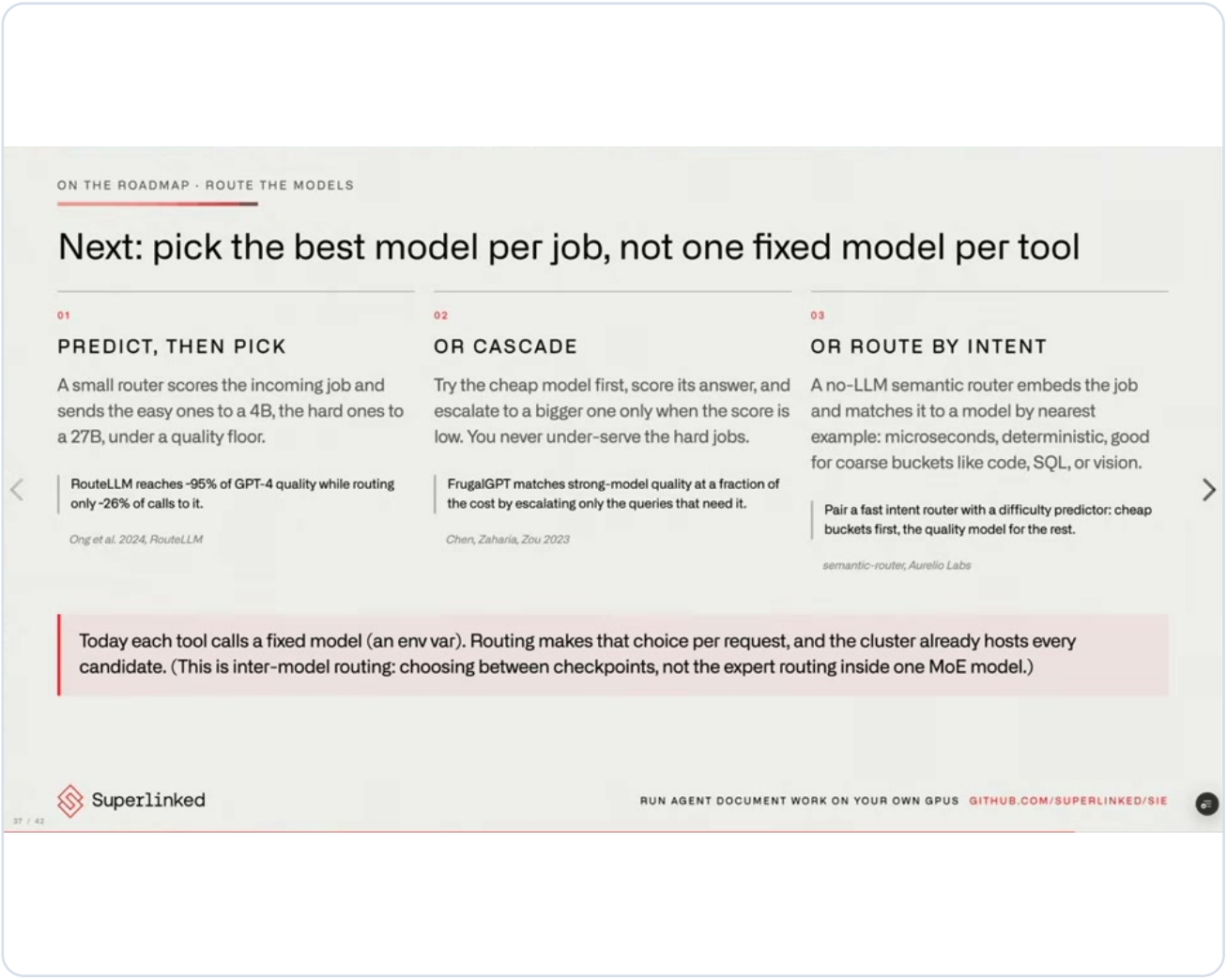
- The edge-and-cluster pattern can process customer calls, transcripts, images, graph data, and ontology-building tasks.
- The edge remains the controlled front door while specialized open models perform each transformation behind it.
- Broader modality support increases capability, but also expands the attack surface, data-retention questions, and evaluation burden.

CORE IDEA

The architecture is a general private inference layer, not only an OCR service.

Future routing chooses a model per request

Predict, cascade, or classify intent rather than fixing one model per tool



NOTES

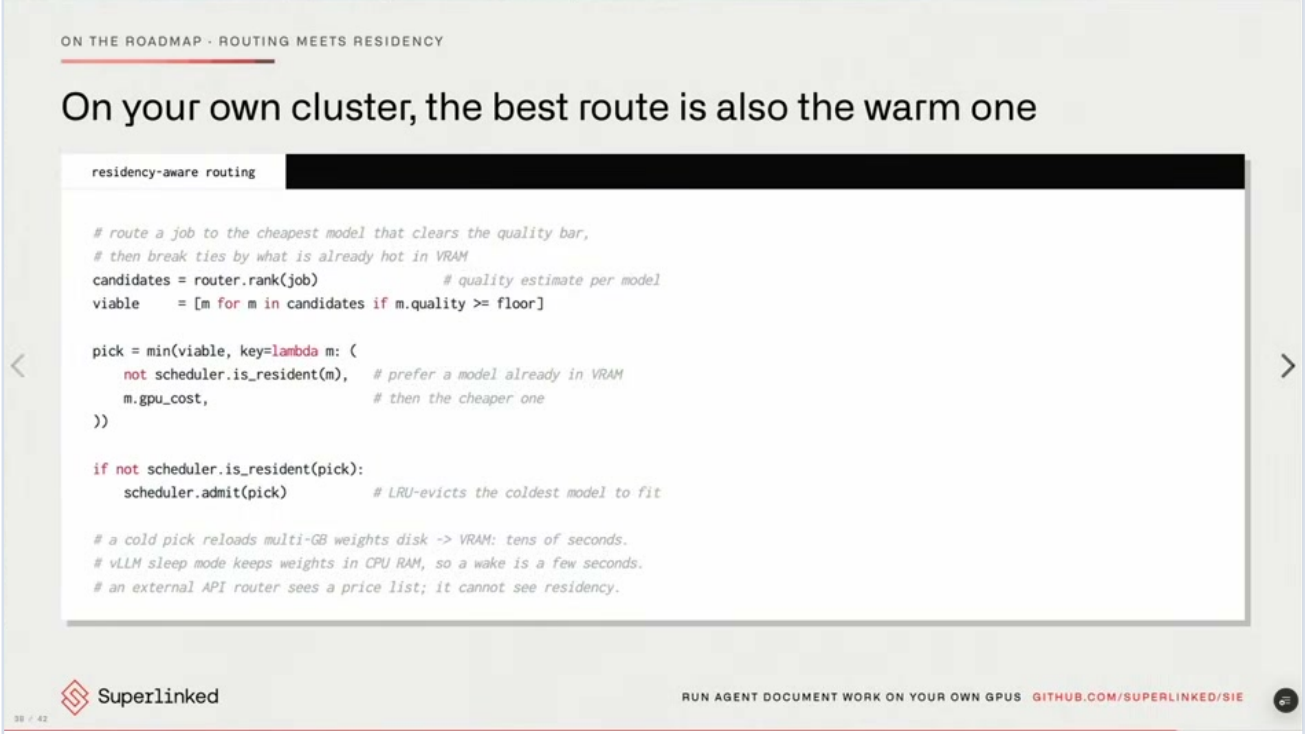
- A predictor can select a model directly, a cascade can escalate only when needed, or intent classification can route by request type.
- This differs from a mixture-of-experts model because routing occurs across separately deployable checkpoints at the system level.
- The talk labels this direction as roadmap work rather than a fully delivered feature.

CORE IDEA

Model routing can optimize quality and cost, but it must be evaluated as a system.

The best route may be the warm model

Loading delay can outweigh a cheaper model's nominal inference price



NOTES

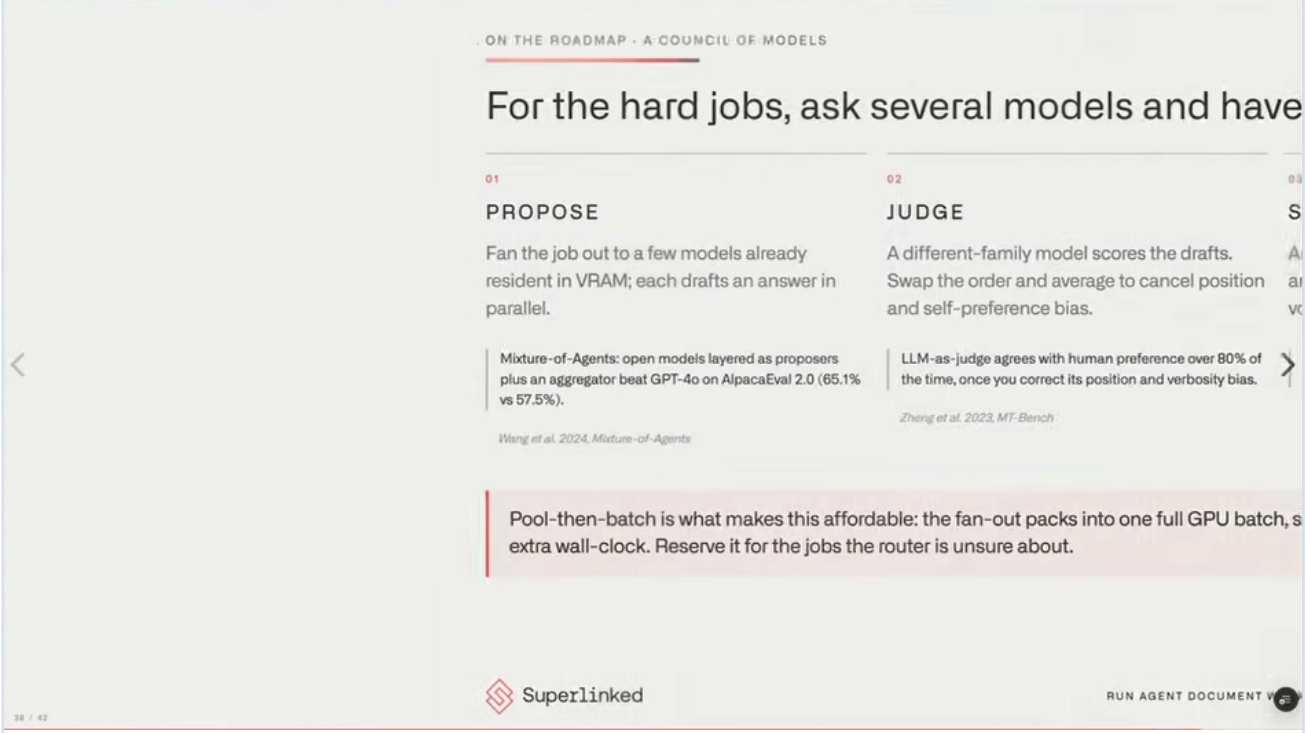
- Routing can include model residency, recent eviction, queue depth, and expected loading delay alongside accuracy and price.
- A nominally more expensive model may complete sooner and cost less end to end if it is already resident on a GPU.
- This requires live infrastructure telemetry and policies that prevent short-term cache state from overriding quality constraints.

CORE IDEA

Production routing should optimize total job cost and latency, not list price alone.

Hard jobs can use a model council

Multiple proposals, a judge, and a synthesis stage



NOTES

- Several models generate candidate answers, a separate model evaluates them, and a final stage selects or synthesizes the result.
- Councils can improve difficult tasks or add reasoning diversity, but they multiply GPU work and increase latency.
- The approach belongs on uncertain, high-value jobs rather than becoming the default path for every request.

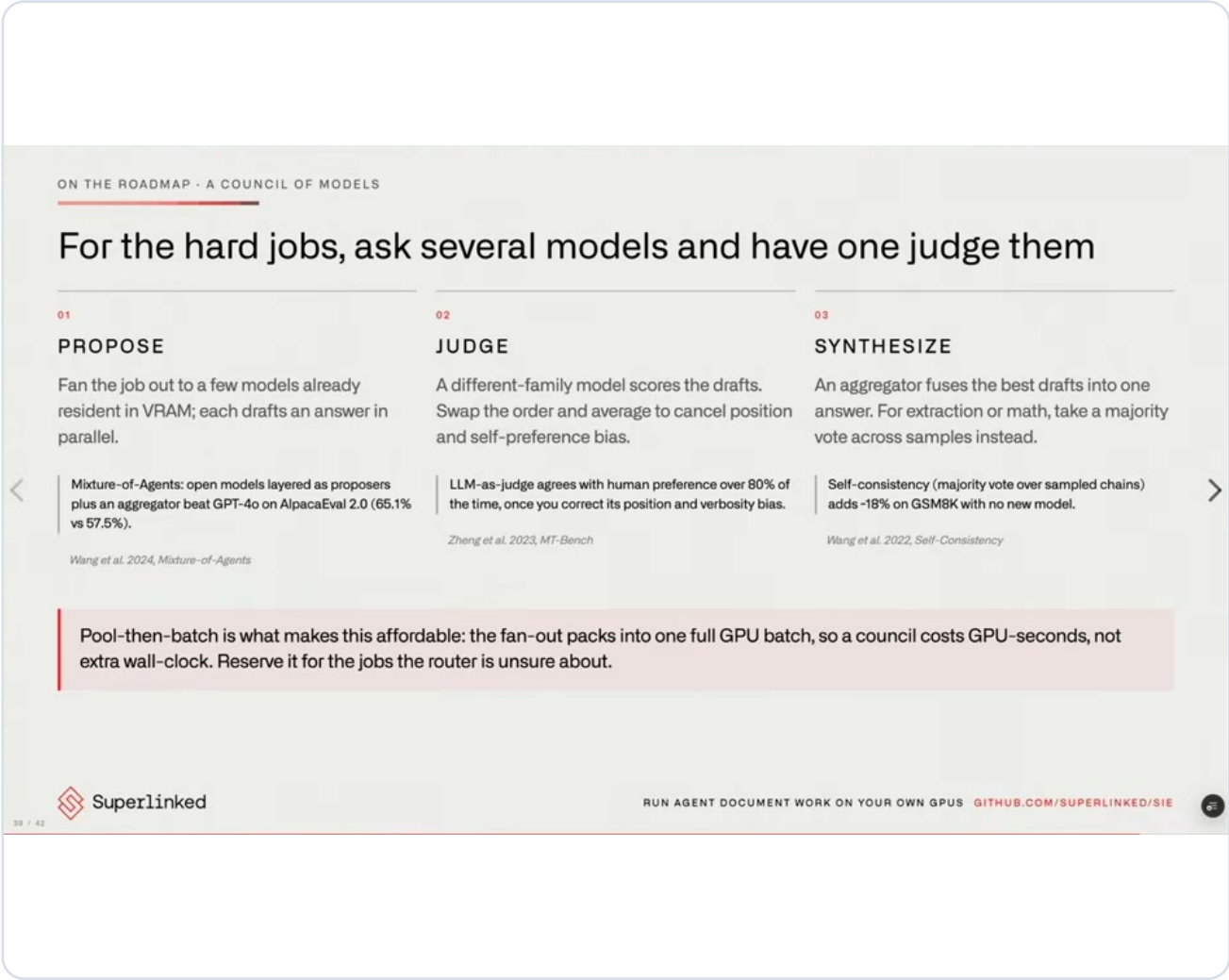
CORE IDEA

Spend ensemble compute selectively where one-model confidence is insufficient.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0

Routers and councils need continuous evaluation

Start with benchmarks, add task and judge tests, then trace production



NOTES

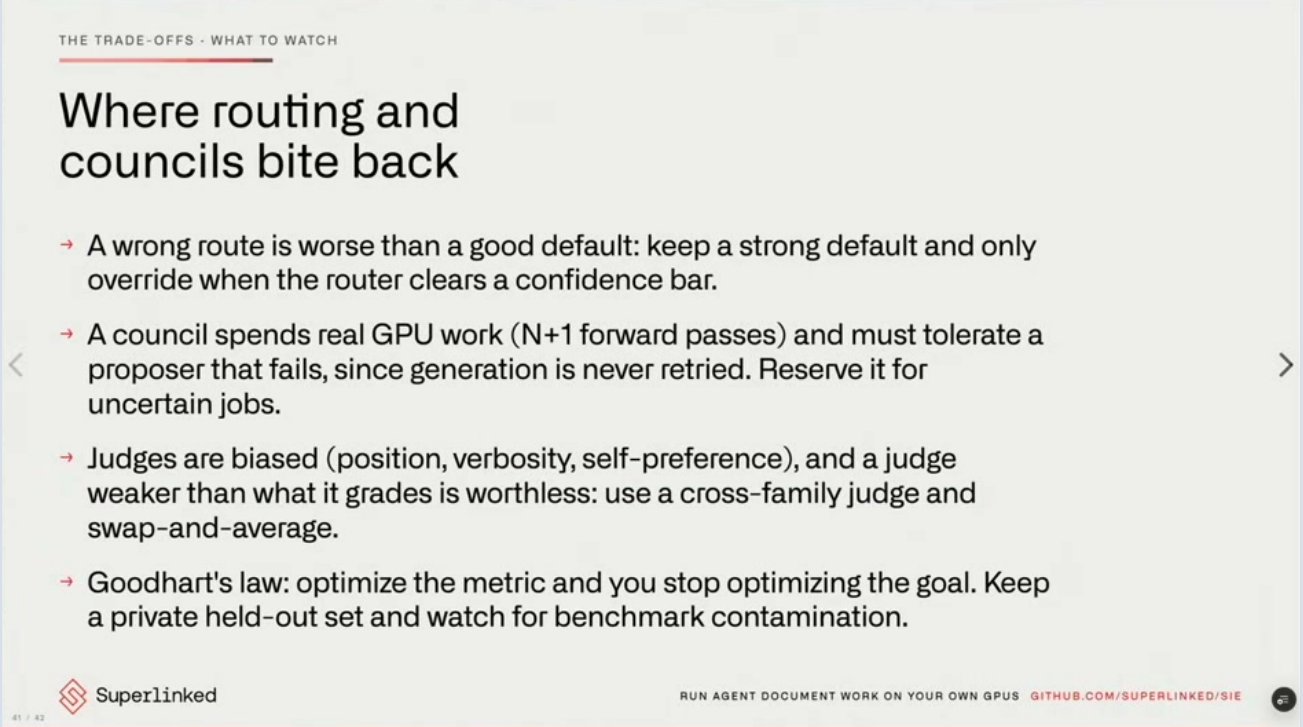
- Massive Text Embedding Benchmark tests provide a baseline but cannot represent every private production task.
- Task-specific and judge evaluations should measure the actual transformations, including redaction misses and retrieval relevance.
- Live tracing detects drift, routing errors, latency regressions, and workload changes after deployment.

CORE IDEA

Routing intelligence is only trustworthy when its decisions are measured continuously.

Where routing and model councils fail

Bad routes, judge bias, excess compute, and Goodhart's law



NOTES

- A weak router can underperform one strong default model, so uncertain choices need confidence thresholds and fallback behavior.
- Judges can prefer their own style, verbosity, or position; cross-family judging and swapped candidate order can reduce some bias.
- Visible benchmarks invite metric gaming and contamination, making private held-out tests and production monitoring necessary.

CORE IDEA

The control system can become the weakest model in the pipeline.

Source: https://www.youtube.com/watch?v=Ndp5VaqFrl0